PROJECT PLAN · v2.0 · 4인 팀

운송서비스 운용 사이트 개발 계획서

Spring Boot + React · 4인 팀 · 기획부터 배포까지 A to Z

26 단계
A ~ Z 상세 작업

10 주
4인 병렬 일정

4 명
도메인별 역할 분담

20 개
데이터베이스 테이블

이 계획서에 포함된 기능

- 배송 접수 · 배차 · 운송 상태 추적
- 차종 · 차량 · 기사 · 근무 일정 관리
- 카카오맵 지도 · 주소 검색
- 상차 · 인수증 사진 업로드
- 리뷰 · 기사 평점
- 공지 · FAQ · 1:1 문의
- 회원 · 역할별 권한 (JWT) · 약관 동의
- 토스페이먼츠 테스트 결제 · 자동 환불
- 거리 · 화물 종류 기반 운임 자동 계산
- 실시간 알림 (SSE) · 기사 위치 추적
- 관리자 통계 · 기사 월 정산
- 상태 이력 타임라인

파란 줄은 MVP에 필요한 핵심 기능, 주황 줄은 확장 기능입니다. 모두 10주 일정에 들어 있습니다.

팀원 1 · 팀장
공통 · 인증 · 운영

팀원 2
기사 · 차량 · 배차 · 정산

팀원 3
배송 접수 · 운임 · 결제

팀원 4
배송 진행 · 실시간 · 리뷰

목차

1 프로젝트 개요 3

2 팀 구성과 역할 분담 NEW 4

3 기술 스택과 개발 환경 5

4 시스템 구성도 7

5 요구사항 정의 8

6 배송 상태와 업무 흐름 10

7 데이터베이스 설계 (ERD · 20개 테이블) 12

8 API 명세 16

9 화면 설계 19

10 프로젝트 폴더 구조 21

11 팀 협업 규칙 NEW 22

12 전체 일정 (10주 · 4인 병렬) 24

13 A ~ Z 단계별 상세 작업 25

- Phase 1 · 기획 (A ~ D) 25
- Phase 2 · 설계 · 준비 (E ~ H) 26
- Phase 3 · 공통 기반 (I ~ K) 27
- Phase 4 · 핵심 기능 (L ~ P) 28
- Phase 5 · 확장 기능 (Q ~ V) 31
- Phase 6 · 마무리 (W ~ Z) 33

14 Claude Code 활용 가이드 35

15 리스크와 대응 36

16 시연 시나리오와 최종 체크리스트 38

17 참고 자료 39

이 문서 사용법

먼저 **2장에서 팀원 번호에 이름을 채우고**, 12장 일정표대로 각자 13장의 A ~ Z 단계 중 자기 담당 단계를 진행하세요. 단계마다 담당자 표시(전원 1 2 3 4)가 있습니다.

4인 팀 개발 원칙

- **도메인별 세로 분담:** 한 사람이 자기 도메인의 테이블 → API → 화면을 끝까지 맡습니다. 백엔드 · 프론트로 나누면 서로 기다리는 시간이 생깁니다.
- **약속 먼저, 구현은 나중에:** 다른 도메인이 부를 메서드는 이름 · 파라미터부터 합의하고 빈 구현으로 먼저 올립니다 (2.4).
- **공통 파일은 PR로만:** SecurityConfig, application.yml, 라우터 같은 공통 파일은 담당자 승인을 받아 수정합니다 (11장).
- **6주차까지 MVP 통합:** "신청 → 결제 → 배차 → 운송 → 완료"가 develop 브랜치에서 한 번에 돌아가야 합니다.

1 프로젝트 개요

1.1 프로젝트 소개

화물 운송 회사가 **배송 접수부터 결제, 배차, 운송 추적, 정산까지** 한 곳에서 처리하는 웹 서비스입니다. 고객(화주)은 온라인으로 운송을 신청하고 예상 운임을 확인한 뒤 결제합니다. 관리자는 기사의 근무 일정과 평점을 고려해 배차하고, 기사는 운송 단계마다 상태 · 사진 · 위치를 공유합니다. 고객은 지도에서 기사 위치를 실시간으로 보고, 배송이 끝나면 리뷰를 남깁니다.

1.2 목표

- 신청 → 결제 → 배차 → 운송 → 완료 → 리뷰 · 정산까지 끊기지 않는 업무 흐름을 구현합니다.
- 지도, 운임 자동 계산, 테스트 결제, 사진 증빙, 실시간 알림 · 위치, 통계, 정산까지 실제 서비스 수준으로 만듭니다.
- 4명이 도메인을 나눠 병렬로 개발하고, PR 리뷰로 코드 품질과 스타일을 맞춥니다.
- 수업(mallapi)의 Spring Boot · React 구조를 확장해 적용하고, AWS에 HTTPS로 배포해 URL로 시연합니다.

1.3 사용자 역할

역할	누구인가	주요 기능
고객 `CUSTOMER`	운송을 맡기는 개인 · 업체 (화주)	배송 신청 · 결제, 예상 운임 확인, 타임라인 · 실시간 위치 · 사진 확인, 취소 · 환불, 리뷰, 1:1 문의
기사 `DRIVER`	차량을 운행하는 기사 (관리자 승인 후 활동)	근무 일정 등록, 내 배차 확인, 단계별 상태 변경, 상차 · 인수증 사진, 위치 공유, 실적 · 평점 확인
관리자 `ADMIN`	운송 회사 운영자	배차, 차종 · 화물 종류 · 차량 · 기사 관리, 결제 · 환불 확인, 통계, 월 정산, 공지 · FAQ · 약관 · 문의 관리

1.4 핵심 업무 흐름

배송 신청 → **운임 자동 계산** → **결제 (테스트)** → **관리자 배차** → **상차 (사진)** → **운송중 (위치 공유)** → **도착** → **배송완료 (인수증)** → **리뷰** → **월말 정산**

단계가 바뀔 때마다 상태 이력이 저장되고, 관련된 사람에게 실시간 알림이 갑니다. 자세한 규칙은 6장에 있습니다.

1.5 범위

포함 (이번 프로젝트에서 구현)	제외 (시간이 남으면 추가 도전 과제)
• 고객 · 기사 · 관리자 웹 화면 전체 (기사 화면은 모바일 웹) • REST API, JWT 인증 · 역할별 권한, 약관 동의 • 카카오맵 지도 · 주소 검색, 거리 · 화물 종류 기반 운임 계산 • 토스페이먼츠 **테스트** 결제 · 취소(환불) • 사진 업로드, 실시간 알림 · 기사 위치 추적(SSE) • 리뷰 · 평점, 통계 대시보드, 월 정산, 공지 · FAQ · 문의 • AWS 배포 (HTTPS)	• 실제 결제 (라이브 키 · 가맹 계약 필요) • 네이티브 모바일 앱, 앱 백그라운드 위치 추적 • 여러 경유지를 거치는 배송, 한 배송에 여러 화물 • 공휴일 할증, 기업 고객 계약 단가 • 프로모션 · 배너

2 팀 구성과 역할 분담

4명이 각자 **하나의 도메인을 테이블부터 화면까지** 맡습니다. 테이블 수가 아니라 작업량으로 나눴고, 발표 때 각자 설명할 **핵심 로직이 하나씩** 있도록 했습니다.

2.1 역할 분담표

담당 (이름)	테이블	주요 화면	핵심 로직 (발표 포인트)
팀원 1 · 팀장 **공통 · 인증 · 운영** 이름: _____	`member` `term` `term_agreement` `notice` `faq` `inquiry` (6개)	로그인 · 회원가입(약관), 마이페이지, 공지 · FAQ · 문의, 관리자 회원 · 약관 관리, 관리자 대시보드	공통 골격 · JWT 보안, 통계 집계 쿼리, HTTPS 배포
팀원 2 **기사 · 차량 · 배차 · 정산** 이름: _____	`driver` `vehicle` `vehicle_type` `driver_schedule` `dispatch` `settlement` (6개)	기사 승인, 차종 · 요금 · 차량 관리, 기사 근무 일정, 배차 화면, 기사 배차 목록 · 실적, 정산	배차 후보 추천 (적재량 · 근무 일정 · 평점), 월 정산 계산
팀원 3 **배송 접수 · 운임 · 결제** 이름: _____	`shipment` `cargo_type` `payment` (3개 · 외부 연동이 많음)	배송 신청(주소 · 지도 · 예상 운임), 결제 성공 · 실패, 내 배송 목록 · 상세, 관리자 배송 검색 · 결제 내역, 화물 종류 관리	카카오 API 연동 · 운임 계산, 결제 승인 · 환불, 결제대기 자동 취소
팀원 4 **배송 진행 · 실시간 · 리뷰** 이름: _____	`shipment_status_history` `shipment_image` `location_log` `notification` `review` (5개)	기사 배송 처리(상태 · 사진 · 위치), 고객 타임라인 · 실시간 지도, 알림 헤더, 리뷰 작성 · 기사 평점	상태 전이 규칙, SSE 실시간 알림 · 위치, 파일 업로드

> **누가 어떤 역할을 맡으면 좋을까**
> **팀원 1**은 모든 팀원이 그 위에서 작업하는 공통 코드와 보안을 맡으므로 Spring에 가장 익숙한 사람이 좋습니다. **팀원 3**은 외부 API(카카오 · 토스) 문서를 읽는 일이 많고, **팀원 4**는 브라우저 기능(SSE · 위치 · 카메라)을 많이 다룹니다. **팀원 2**는 조건이 많은 쿼리와 계산 로직이 중심입니다.

2.2 공통 책임

역할	담당	하는 일
팀장 · 일정 관리	1	주간 회의 진행, 이슈 보드 관리, 주차별 완료 기준 점검
배포 · 인프라	1	AWS · Nginx · HTTPS, 환경변수 · 키 관리, 배포 문서
Git · ERD 관리	2	브랜치 보호 설정, develop → main 병합, ERD 원본과 `docs/erd.md` 관리, 시드 데이터 (`data.sql`)
API 문서 관리	3	Swagger(springdoc) 설정, `docs/api.md` · Postman 컬렉션 최신화
UI 통일 관리	4	색 · 상태 배지 · 공통 컴포넌트 규칙, 9주차 화면 점검 주도

2.3 도메인 간 호출 관계

도메인끼리는 서로의 **Service 메서드**로만 호출합니다. 다른 사람의 Repository나 엔티티 내부를 직접 고치지 않습니다.

팀원 3 · 배송 신청 · 결제 → 배차 대기 → **팀원 2 · 배차** → 배정 기사 → **팀원 4 · 상태 · 위치 · 리뷰** → 완료 건 · 평점 → **팀원 2 · 정산** → **팀원 1 · 통계**

2.4 도메인 간 약속 (인터페이스)

언제까지	제공 → 받는 사람	약속 내용
2주차 말	1 → 전원	공통 골격 1차: BaseEntity, 예외 · 에러 응답 형식, PageRequestDTO / PageResponseDTO, CORS, 개발용 임시 보안 설정(전체 허용)
3주차 초	1 → 전원	로그인한 사용자 정보를 꺼내는 방법(예: `@AuthenticationPrincipal`), 역할별 URL 규칙
3주차 중	3 → 2 4	`Shipment` 엔티티 · `ShipmentStatus` enum · Repository (필드는 7장 그대로)
3주차 중	2 → 3	`VehicleType` 엔티티와 초기 데이터 (요금 컬럼 포함)
4주차	4 → 전원	`NotificationService.send(받는 회원, 유형, 메시지, 배송)` — 처음에는 로그만 찍는 빈 구현
5주차	2 → 4	`DispatchService.getActiveDriver(shipmentId)` — 상태 변경 · 위치 전송 권한 확인용
5주차	3 → 전원	개발용 "결제 건너뛰기" API (local 프로필에서만 동작) — 결제 없이 접수 상태를 만들어 다른 기능을 테스트
8주차	4 → 2	`ReviewService.getAverageRating(driverId)` — 배차 후보 정렬 · 기사 실적용

약속을 지키는 방법

메서드 이름 · 파라미터 · 반환값만 먼저 합의하고 **빈 구현을 develop에 올리세요.** 그러면 실제 구현이 끝나기 전에도 다른 팀원이 그 메서드를 호출하며 개발할 수 있습니다. 약속이 늦어지면 바로 데일리 스크럼에서 알립니다.

3 기술 스택과 개발 환경

구분	기술	용도 · 비고
백엔드 (STS)	Java 17 이상, Spring Boot 3.x	수업(mallapi)과 **같은 버전** (메모 ①), 팀 전원 같은 JDK
	Gradle	빌드 도구
	Spring Data JPA (Hibernate)	DB 접근. 관리자 복합 검색은 QueryDSL (수업에서 다뤘다면)
	Spring Security + JWT, Validation	로그인 · Access/Refresh 토큰 · 역할별 접근 제어, `@Valid` 요청 검증
	SseEmitter, `@Scheduled`	실시간 알림 · 위치 전달, 결제대기 자동 취소 · 오래된 위치 기록 삭제
	RestClient (또는 RestTemplate)	카카오 · 토스페이먼츠 API 서버 호출
	springdoc-openapi (Swagger UI)	팀이 함께 보는 API 문서 자동 생성 — Spring Boot 버전에 맞는 springdoc 버전 사용
	Thumbnailator, Lombok, ModelMapper	사진 썸네일, 코드 간소화, DTO 변환 (수업 프로젝트에 있는 라이브러리 우선)
	MariaDB 또는 MySQL 8	수업과 같은 DB, 팀원 전원 로컬에 같은 버전 설치

구분	기술	용도 · 비고
프론트엔드 (VS Code)	React + Vite, React Router	수업과 같은 버전, 역할별 보호 라우트
	Redux Toolkit, axios	로그인 정보 전역 관리(수업의 loginSlice), 인터셉터로 JWT 첨부 · 재발급
	Tailwind CSS	화면 스타일 (팀원 4가 색 · 컴포넌트 규칙 관리)
	Kakao Maps JavaScript SDK, `react-daum-postcode`	지도 · 마커 · 기사 위치 표시, 주소 검색 팝업
	토스페이먼츠 JavaScript SDK	테스트 결제창 열기 (메모 ④)
	Geolocation API, `event-source-polyfill`	기사 휴대폰 위치 받기(브라우저 내장, 메모 ⑤), 헤더에 토큰을 실을 수 있는 SSE 연결
	Recharts	대시보드 차트
외부 서비스	카카오 로컬 API · 카카오모빌리티 길찾기 API	주소 → 좌표, 도로 거리 · 소요시간 (서버에서 REST API 키로 호출, 비상업 용도는 무료 쿼터 안에서 별도 제휴 없이 사용)
	토스페이먼츠 (테스트 키)	결제 승인 · 취소. 테스트 키로는 실제 돈이 빠져나가지 않음
	AWS (EC2 · RDS · S3)	서버 · DB · 이미지 저장 (배포 단계)
협업 도구	GitHub (저장소 · PR · Projects 이슈 보드), 메신저(디스코드 · 카카오톡), Postman 팀 워크스페이스, ERDCloud, Figma, Claude Code, HTTPS 터널 도구(예: ngrok — 휴대폰 위치 테스트)	

메모 ① Spring Boot 버전은 수업과 맞추고, 팀 전원이 같은 버전을 쓰세요

Spring Boot 4.0이 2025년 11월에 정식 출시됐지만 Spring Security 7 · Jackson 3 · Hibernate 7 등 기반 라이브러리가 함께 바뀌어 3.x 코드를 그대로 옮길 수 없습니다. 수업과 같은 **3.x 버전으로 고정**하고, JDK · DB 버전까지 팀 전원이 맞추세요. CLAUDE.md에도 버전을 적어 둡니다.

메모 ② 카카오맵 API 무료 쿼터는 "계정당 첫 번째 앱"에만

2026년 7월 21일부터 카카오맵 API는 별도 신청 · 심사 없이 앱 관리 페이지에서 활성화하면 쓸 수 있습니다. 다만 **무료 쿼터는 개발자 계정 기준으로 처음 활성화한 앱 하나에만** 주어지고, 두 번째 앱부터는 비즈월렛 연결과 유료 API 설정이 필요합니다. **팀에서 카카오 앱은 한 사람(팀원 3)의 계정으로 하나만** 만들고, 다른 팀원은 팀원으로 초대받거나 키를 공유받아 씁니다.

메모 ③ AWS 무료 플랜은 6개월 · 크레딧 방식

2025년 7월 15일 이후 만든 AWS 계정은 크레딧(가입 100달러 + 활동 시 최대 100달러) 기반 무료 플랜이 적용되며, **가입 후 6개월 또는 크레딧 소진 중 빠른 시점**에 끝나고 유료 전환을 하지 않으면 계정이 닫힙니다(데이터는 90일 보관). 계정은 배포 담당(팀원 1)이 8주차에 만들고 종료일을 공유합니다.

메모 ④ 토스페이먼츠 테스트 결제

계약하지 않아도 개발자센터에 **회원가입만 하면 "개발자용 테스트상점" 테스트 키**가 발급되고, 테스트 키로 결제하면 실제 결제가 일어나지 않습니다. 결제위젯 연동 키는 전자결제 신청(사업자 정보)을 요구했다는 사례가 있으니, **API 개별 연동 키로 결제창을 여는 방식**을 기본으로 합니다. 클라이언트 키와 시크릿 키는 반드시 같은 세트를 써야 하며, 시크릿 키는 서버 설정 파일에만 둡니다.

메모 ⑤ 기사 위치 추적은 HTTPS에서만 동작

브라우저 위치(Geolocation) API는 **HTTPS 또는 localhost 페이지**에서만 동작합니다. 휴대폰으로 `http://192.168.x.x:5173` 같은 개발 서버에 접속하면 위치를 받을 수 없으므로, 개발 중에는 HTTPS 터널로 테스트하고 **배포는 HTTPS가 필수**입니다.

4 시스템 구성도

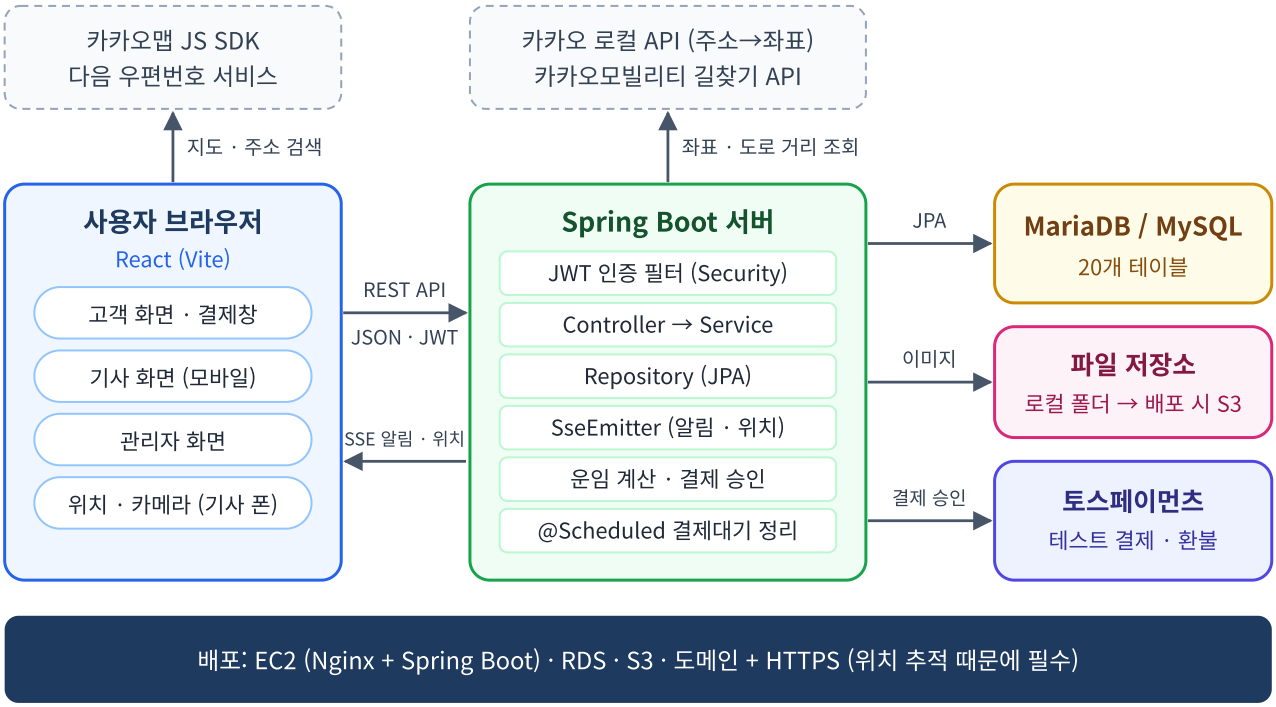


그림 1. 시스템 구성도 — 외부 API 호출 · 운임 계산 · 결제 승인은 서버에서, 지도 · 결제창 · 위치 받기는 브라우저에서 처리합니다.

4.1 요청 흐름 예시: 배송 신청부터 접수까지

1. 신청 화면이 `POST /api/shipments`를 보냅니다. 서버는 좌표 · 거리 · 운임을 직접 다시 계산하고, 배송을 **결제대기**로, 결제 정보(주문번호 · 금액)를 **READY**로 저장합니다.
2. 화면은 응답받은 주문번호 · 금액으로 **토스페이먼츠 결제창**(테스트)을 엽니다.
3. 결제에 성공하면 토스가 successUrl(우리 결제 성공 페이지)로 돌려보내고, 이 페이지가 `POST /api/payments/confirm`을 호출합니다.
4. 서버는 **저장해 둔 금액과 요청 금액이 같은지** 확인한 뒤 토스 승인 API를 호출하고, 성공하면 결제 PAID · 배송 **접수**로 바꾸고 관리자에게 알립니다.
5. 결제하지 않고 30분이 지나면 스케줄러가 배송을 자동으로 취소합니다.

4.2 처리 위치 원칙

처리 위치	담당하는 일
브라우저 (React)	화면 표시, 입력 검증(1차), 지도 · 마커, 주소 검색, 결제창 열기, 기사 위치 받기 · 사진 촬영, 차트 · 알림 표시
서버 (Spring Boot)	권한 검사, 입력 검증(최종), 상태 전이 규칙, 운임 계산, 결제 금액 검증 · 승인 · 취소, 외부 API 호출(키 보호), 집계 쿼리, 파일 저장, 알림 · 위치 전달, 스케줄러
DB	모든 데이터 저장, 유니크 · 외래키 제약으로 데이터 무결성 보장

5 요구사항 정의

기능마다 ID와 담당자를 붙여 API 명세 · 화면 · 테스트 · 이슈에서 같은 번호로 추적합니다. 핵심 은 MVP에 꼭 필요한 기능, 확장 은 MVP 이후 붙이는 기능입니다.

5.1 공통

ID	기능	상세	구분	담당
COM-01	회원가입	이메일 · 비밀번호 · 이름 · 연락처, 역할 선택(고객/기사), 필수 약관 동의(동의 이력 저장). 기사는 면허 · 경력 입력 후 관리자 승인	핵심	1
COM-02	로그인 · 로그아웃	JWT 발급, 만료 시 Refresh 토큰으로 자동 재발급, 역할별 첫 화면으로 이동	핵심	1
COM-03	내 정보	연락처 · 비밀번호 변경	핵심	1
COM-04	알림	배차 · 상태 변경 · 문의 답변 시 실시간 알림(SSE), 목록 · 읽음 처리, 안 읽은 개수 배지	확장	4
COM-05	공지사항	목록 · 상세 (작성은 관리자)	확장	1
COM-06	FAQ	카테고리별 자주 묻는 질문	확장	1
COM-07	1:1 문의	문의 등록, 내 문의 목록, 답변 확인	확장	1

5.2 고객

ID	기능	상세	구분	담당
CUS-01	배송 신청	출발지 · 도착지 주소 검색, 지도 마커, 화물 종류 · 화물명 · 무게 · 희망 상차일시 · 수령인 · 요청사항 입력	핵심	3
CUS-02	예상 운임	거리 · 차종 · 화물 종류 · 할증을 반영한 운임과 계산 내역 표시, 무게에 맞는 차종 자동 추천	확장	3
CUS-03	결제	토스페이먼츠 테스트 결제창, 실패 시 재시도, 30분 안에 결제하지 않으면 자동 취소	핵심	3
CUS-04	내 배송 목록	상태 탭, 기간 검색, 페이징	핵심	3
CUS-05	배송 상세 · 추적	상태 타임라인, 기사 · 차량 정보, 사진, **운송중일 때 기사 위치 실시간 지도**	핵심	3 4
CUS-06	취소 · 환불	'접수' 상태에서 취소하면 결제 자동 취소(환불), 취소 사유 입력	핵심	3
CUS-07	리뷰	배송완료 후 1회 별점(1 ~ 5) · 후기 작성	확장	4

5.3 기사

ID	기능	상세	구분	담당
DRV-01	근무 일정	날짜별 근무 가능 시간 · 휴무 등록 (달력)	확장	2
DRV-02	내 배차 목록	오늘 / 예정 / 완료 탭, 카드형 목록 (휴대폰 화면 기준)	핵심	2
DRV-03	배송 처리 화면	지도, 화물 · 수령인 정보, 현재 상태에 맞는 "다음 단계" 버튼, 수령인 전화 걸기	핵심	4
DRV-04	상태 변경	상차완료 → 운송중 → 도착 → 배송완료 순서로만 변경	핵심	4
DRV-05	사진 업로드	상차 사진(상차완료 전 필수), 인수증 사진(배송완료 전 필수), 휴대폰 카메라 바로 열기	확장	4

ID	기능	상세	구분	담당
DRV-06	위치 공유	운송중 동안 휴대폰 위치를 30초 간격으로 전송, 도착 처리 시 자동 중지	확장	4
DRV-07	실적 · 평점	월별 완료 건수 · 운임 합계 · 정산 상태 · 평균 평점 · 받은 리뷰	확장	2

5.4 관리자

ID	기능	상세	구분	담당
ADM-01	대시보드	오늘 접수 · 진행 · 완료 건수, 이번 달 매출, 평균 평점, 일별 추이 · 차종별 차트, 기사 순위	확장	1
ADM-02	배송 관리	전체 배송 검색(상태 · 기간 · 고객 · 기사 · 키워드), 상세, 취소(자동 환불)	핵심	3
ADM-03	결제 내역	결제 · 환불 내역 조회 (상태 · 기간)	확장	3
ADM-04	배차	후보 추천(차종 · 적재량 · 근무 일정 · 당일 배차 수, 평점순 정렬), 배정 · 해제	핵심	2
ADM-05	기사 관리	가입 승인, 활동 · 중지, 정보 · 평점 조회	핵심	2
ADM-06	차종 · 요금	차종별 최대 적재량 · 기본요금 · km당 요금 · 할증률 수정	확장	2
ADM-07	차량 관리	차량 등록 · 수정 · 비활성화, 차종 선택, 담당 기사 지정	핵심	2
ADM-08	화물 종류	일반 · 파손주의 등 추가요금 · 취급 안내 관리	확장	3
ADM-09	정산	월 단위 기사별 운임 합계 · 수수료 · 지급액 계산, 지급 처리	확장	2
ADM-10	회원 관리	회원 검색, 역할 확인, 비활성화	핵심	1
ADM-11	공지 · FAQ · 약관 · 문의	공지 · FAQ · 약관 작성 · 수정, 문의 답변 (답변 시 알림)	확장	1

5.5 비기능 요구사항

항목	기준
보안	비밀번호 BCrypt 암호화, 역할별 API 접근 제어, 본인 · 배정 기사 · 관리자만 배송 상세 조회, API 키 · 시크릿 키 · DB 비밀번호는 설정 파일 · 환경변수로 분리
결제 안전	운임은 서버에서만 계산, 승인 전 저장된 금액과 요청 금액 비교, 승인 성공 후 저장이 실패하면 결제 취소로 되돌림
개인정보	기사 위치는 운송중일 때만 수집 · 표시, 배송완료 후 고객 화면에서 숨김, 30일 지난 위치 기록은 스케줄러로 삭제
데이터 무결성	허용된 상태 전이만 가능, 동시 배차 · 동시 상태 변경 방지(`@Version`)
성능	목록은 페이징(기본 10건), N+1 쿼리 방지, 같은 출발 · 도착 구간의 거리 결과 재사용, 위치는 30초 간격
사용성 · 시간대	기사 화면 모바일 우선, 로딩 · 빈 목록 · 에러 표시, 금액 천 단위 쉼표 / 서버 · DB · JDBC 모두 `Asia/Seoul`
협업 품질	모든 코드는 PR 리뷰 1명 이상 승인 후 병합, develop 브랜치는 항상 실행 가능한 상태 유지

6 배송 상태와 업무 흐름

배송 상태는 4명이 함께 쓰는 **가장 중요한 규칙**입니다. 결제가 들어오면서 맨 앞에 **결제대기** 상태가 추가됐습니다. 1주차에 전원이 함께 확정합니다.

6.1 상태 정의

상태 (enum)	한글	의미	누가 바꾸나
`PENDING_PAYMENT`	결제대기	신청서가 저장됐고 결제를 기다리는 중 (관리자 배차 목록에는 안 보임)	고객 (신청)
`REQUESTED`	접수	결제 완료, 배차 대기	시스템 (결제 승인)
`ASSIGNED`	배차완료	기사 · 차량 배정됨	관리자
`PICKED_UP`	상차완료	화물을 차에 실음 (상차 사진 첨부)	배정된 기사
`IN_TRANSIT`	운송중	도착지로 이동 중 — 이때만 위치 공유	배정된 기사
`ARRIVED`	도착	도착지 도착, 인계 전	배정된 기사
`COMPLETED`	배송완료	인계 완료 (인수증 사진 필수). 이후 변경 불가, 리뷰 가능	배정된 기사
`CANCELED`	취소	결제대기 · 접수 · 배차완료 단계에서 취소 (결제된 건은 자동 환불)	고객 · 관리자 · 시스템

6.2 상태 다이어그램

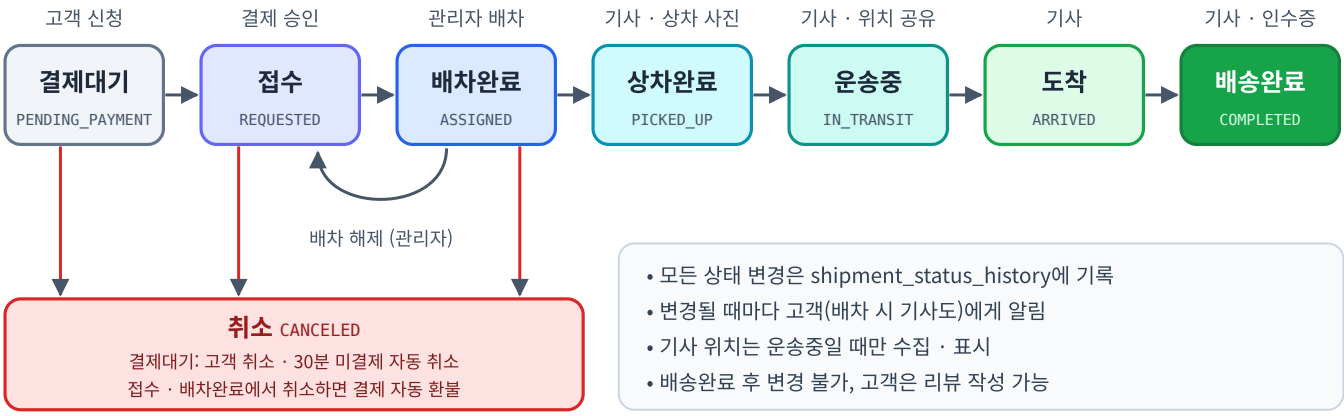


그림 2. 배송 상태 전이 — 박스 위 글자는 그 상태로 바꾸는 주체와 조건입니다.

6.3 전이 규칙

현재 → 다음	누가	조건	함께 처리할 일	담당
(신규) → 결제대기	고객	신청서 검증 통과	운임 계산 · 저장, 결제 READY 생성	3
결제대기 → 접수	시스템	토스 승인 성공 + 금액 일치	결제 PAID, 관리자 알림	3
결제대기 → 취소	고객 · 시스템	고객 취소, 또는 30분 경과(스케줄러)	결제 EXPIRED	3

현재 → 다음	누가	조건	함께 처리할 일	담당
접수 → 배차완료	관리자	기사 ACTIVE, 근무 일정 안, 차량 AVAILABLE, 적재량 ≥ 무게, 당일 배차 수 < 한도	배차 생성, 고객 · 기사 알림	2
배차완료 → 접수	관리자	배차 해제	배차 active = false	2
배차완료 → 상차완료	배정된 기사	상차 사진 1장 이상	고객 알림	4
상차완료 → 운송중	배정된 기사	—	위치 공유 시작, 고객 알림	4
운송중 → 도착	배정된 기사	—	위치 공유 중지, 고객 알림	4
도착 → 배송완료	배정된 기사	인수증 사진 1장 이상	고객 알림 (리뷰 요청)	4
접수 → 취소	고객 · 관리자	취소 사유 입력	결제 취소(환불), 알림	3
배차완료 → 취소	관리자	취소 사유 입력	배차 비활성화, 환불, 고객 · 기사 알림	3

모든 전이에서 공통으로 `shipment_status_history`에 (이전 상태, 다음 상태, 변경자, 사유, 시각)을 저장합니다. 이력 저장 메서드는 팀원 4가 만들고 나머지 팀원이 호출합니다.

6.4 구현 방법: 규칙을 enum 한 곳에 모으기

```
public enum ShipmentStatus {
    PENDING_PAYMENT, REQUESTED, ASSIGNED, PICKED_UP, IN_TRANSIT, ARRIVED, COMPLETED, CANCELED;

    // 상태별로 갈 수 있는 다음 상태 (COMPLETED, CANCELED는 끝 상태)
    private static final Map<ShipmentStatus, Set<ShipmentStatus>> NEXT = Map.of(
        PENDING_PAYMENT, Set.of(REQUESTED, CANCELED),
        REQUESTED,       Set.of(ASSIGNED, CANCELED),
        ASSIGNED,        Set.of(PICKED_UP, REQUESTED, CANCELED),
        PICKED_UP,       Set.of(IN_TRANSIT),
        IN_TRANSIT,      Set.of(ARRIVED),
        ARRIVED,         Set.of(COMPLETED)
    );

    public boolean canChangeTo(ShipmentStatus next) {
        return NEXT.getOrDefault(this, Set.of()).contains(next);
    }
}

// Shipment 엔티티 안의 변경 메서드 — "누가" 검사는 각 Service에서, "갈 수 있나" 검사는 여기서
public void changeStatus(ShipmentStatus next) {
    if (!this.status.canChangeTo(next)) {
        throw new CustomException(ErrorCode.INVALID_STATUS_CHANGE);
    }
    this.status = next;
}
```

7 데이터베이스 설계 (ERD · 20개 테이블)

v1의 12개 테이블에 결제 · 화물 종류 · 근무 일정 · 위치 기록 · 리뷰 · 약관 · FAQ를 더하고, 요금 규칙을 차종 테이블로 합쳤습니다. 담당자별 색으로 묶었습니다.

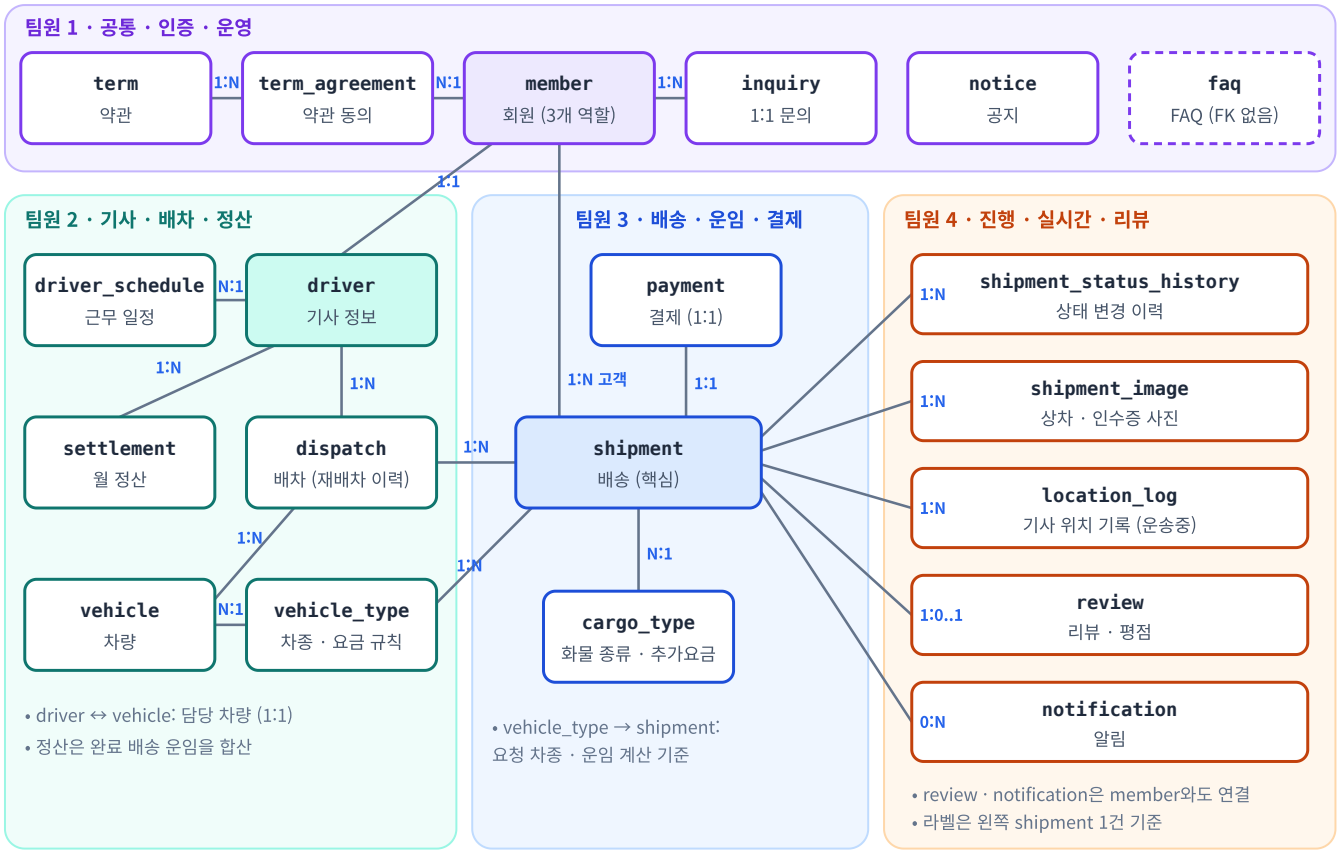


그림 3. 담당자별 테이블 지도 (20개) — 선은 주요 외래키 관계만 표시했습니다. 전체 컬럼은 7.1 ~ 7.5에 있습니다.

7.1 배송 테이블 (모두가 참조하는 중심 테이블)

공통: 모든 테이블은 `id BIGINT PK AUTO_INCREMENT`와 `created_at` · `updated_at`(BaseEntity)을 가집니다. enum은 `@Enumerated(EnumType.STRING)`으로 저장합니다.

`shipment` 배송 · 담당 팀원 3

컬럼	타입	설명
customer_id (FK)	BIGINT	신청 고객 → member
vehicle_type_id (FK)	BIGINT	요청 차종 → vehicle_type (무게로 자동 추천, 고객 변경 가능)
cargo_type_id (FK)	BIGINT	화물 종류 → cargo_type
origin_address / origin_detail	VARCHAR	출발지 주소 / 상세 주소
origin_lat / origin_lng	DOUBLE	출발지 위도 / 경도
dest_address / dest_detail	VARCHAR	도착지 주소 / 상세 주소
dest_lat / dest_lng	DOUBLE	도착지 위도 / 경도
distance_km / duration_min	DOUBLE / INT	도로 거리 / 예상 소요시간(분)

컬럼	타입	설명
cargo_name / cargo_weight_kg	VARCHAR / INT	화물명 / 무게(kg)
pickup_at	DATETIME	희망 상차 일시 (야간 · 주말 할증 기준)
receiver_name / receiver_phone	VARCHAR	수령인 이름 / 연락처
memo / cancel_reason	VARCHAR(500)	요청사항 / 취소 사유
fare	INT	운임 = 결제 금액 (서버가 계산한 값)
status	VARCHAR(20)	배송 상태 (6장, 결제대기 포함)
version	BIGINT	`@Version` 낙관적 락 — 동시 배차 · 동시 상태 변경 방지

팀원 1 공통 · 인증 · 운영 (6개)

member 회원

컬럼	타입	설명
email (UQ)	VARCHAR(100)	로그인 아이디
password	VARCHAR(255)	BCrypt 암호화
name / phone	VARCHAR	이름 / 연락처
role	VARCHAR(20)	CUSTOMER · DRIVER · ADMIN
enabled	BOOLEAN	활성 여부

term 약관

컬럼	타입	설명
title	VARCHAR(100)	약관 이름
content	TEXT	약관 내용
required	BOOLEAN	필수 동의 여부
version	VARCHAR(20)	버전 (예: v1.0)
active	BOOLEAN	현재 사용 여부

term_agreement 약관 동의 이력

컬럼	타입	설명
member_id (FK)	BIGINT	동의한 회원
term_id (FK)	BIGINT	동의한 약관 (회원+약관 UQ)
agreed_at	DATETIME	동의 시각

notice 공지

컬럼	타입	설명
title / content	VARCHAR / TEXT	제목 / 내용
writer_id (FK)	BIGINT	작성 관리자
view_count	INT	조회수

faq 자주 묻는 질문

컬럼	타입	설명
category	VARCHAR(30)	배송 · 결제 · 회원 등
question	VARCHAR(255)	질문
answer	TEXT	답변
sort_order / active	INT / BOOLEAN	정렬 순서 / 노출 여부

inquiry 1:1 문의

컬럼	타입	설명
member_id (FK)	BIGINT	작성 회원
title / content	VARCHAR / TEXT	제목 / 내용
answer / answered_at	TEXT / DATETIME	답변 / 답변일
status	VARCHAR(20)	WAITING · ANSWERED

팀원 2 기사 · 차량 · 배차 · 정산 (6개)

`driver` 기사 정보 (member와 1:1)

컬럼	타입	설명
member_id (FK, UQ)	BIGINT	기사 회원
license_type	VARCHAR(20)	면허 종류
career_years	INT	경력(년)
status	VARCHAR(20)	PENDING · ACTIVE · INACTIVE
approved_at	DATETIME	승인 일시

`vehicle_type` 차종 · 요금 규칙

컬럼	타입	설명
code (UQ) / name	VARCHAR	SMALL · TON1 · TON2_5 · TON5 / 표시 이름
max_load_kg	INT	최대 적재량(kg)
base_fare / base_distance_km	INT	기본요금 / 기본 포함 거리
per_km_fare	INT	초과 1km당 요금
night / weekend / heavy_rate	INT	야간 · 주말 · 고중량 할증률(%)
active	BOOLEAN	사용 여부

`vehicle` 차량

컬럼	타입	설명
plate_no (UQ)	VARCHAR(20)	차량번호
vehicle_type_id (FK)	BIGINT	차종
status	VARCHAR(20)	AVAILABLE · MAINTENANCE · INACTIVE
driver_id (FK, UQ)	BIGINT	담당 기사 (NULL 가능)

`driver_schedule` 근무 일정

컬럼	타입	설명
driver_id (FK)	BIGINT	기사 (기사+날짜 UQ)
work_date	DATE	날짜
start_time / end_time	TIME	근무 가능 시간
status	VARCHAR(20)	AVAILABLE · OFF

`dispatch` 배차 (재배차 이력 보존)

컬럼	타입	설명
shipment_id (FK)	BIGINT	배송
driver_id / vehicle_id	BIGINT	배정 기사 / 차량
assigned_by (FK)	BIGINT	배차한 관리자
active	BOOLEAN	현재 유효한 배차 (배송당 1건)
assigned_at / released_at	DATETIME	배정 / 해제 일시

`settlement` 월 정산

컬럼	타입	설명
driver_id (FK)	BIGINT	기사 (기사+월 UQ)
settle_month	CHAR(7)	정산 월 (예: 2026-12)
completed_count	INT	완료 건수
total_fare / payout_amount	INT	운임 합계 / 지급액
commission_rate	INT	수수료율(%)
status / paid_at	VARCHAR / DATETIME	PENDING · PAID / 지급일

팀원 3 배송 접수 · 운임 · 결제 (shipment 포함 3개)

`cargo_type` 화물 종류

컬럼	타입	설명
name (UQ)	VARCHAR(30)	일반 · 파손주의 · 가구/가전 등
extra_fee	INT	추가요금(원)
handling_note	VARCHAR(255)	기사에게 보여줄 취급 안내
active	BOOLEAN	사용 여부

`payment` 결제 (배송과 1:1)

컬럼	타입	설명
shipment_id (FK, UQ)	BIGINT	배송
order_id (UQ)	VARCHAR(64)	주문번호 (재시도 시 새로 발급)
payment_key	VARCHAR(200)	토스 결제 키 (승인 후 저장)
amount / method	INT / VARCHAR	결제 금액 (승인 전 검증) / 결제 수단
status	VARCHAR(20)	READY · PAID · CANCELED · EXPIRED
paid_at / canceled_at	DATETIME	승인 / 취소 일시 (+ cancel_reason)

팀원 4 배송 진행 · 실시간 · 리뷰 (5개)

`shipment_status_history` 상태 이력

컬럼	타입	설명
shipment_id (FK)	BIGINT	배송
from_status / to_status	VARCHAR(20)	이전 / 다음 상태
changed_by (FK)	BIGINT	변경한 회원 (시스템은 NULL)
memo / changed_at	VARCHAR / DATETIME	사유 / 변경 시각

`shipment_image` 사진

컬럼	타입	설명
shipment_id (FK)	BIGINT	배송
type	VARCHAR(20)	PICKUP · RECEIPT · ETC
file_name	VARCHAR(255)	저장 파일명 (UUID_원본명)
uploaded_by (FK)	BIGINT	올린 기사

`location_log` 기사 위치 기록

컬럼	타입	설명
shipment_id (FK)	BIGINT	배송 (+ recorded_at 인덱스)
driver_id (FK)	BIGINT	보낸 기사
lat / lng	DOUBLE	위도 / 경도
recorded_at	DATETIME	측정 시각 (30일 후 삭제)

`review` 리뷰 · 평점

컬럼	타입	설명
shipment_id (FK, UQ)	BIGINT	배송 (배송당 1개)
customer_id (FK)	BIGINT	작성 고객
driver_id (FK)	BIGINT	평가 대상 기사
rating / content	TINYINT / VARCHAR	별점 1 ~ 5 / 후기 (500자)

notification 알림

컬럼	타입	설명
receiver_id (FK)	BIGINT	받는 회원
shipment_id (FK)	BIGINT	관련 배송 (NULL 가능)
type	VARCHAR(30)	DISPATCHED · STATUS_CHANGED · INQUIRY_ANSWERED · REVIEW_REQUEST
message / is_read	VARCHAR / BOOLEAN	알림 문구 / 읽음 여부

설계 메모

- 연관관계는 `@ManyToOne(fetch = FetchType.LAZY)` 단방향으로 시작하고, 필요한 곳만 양방향으로 엽니다.
- 배송과 기사는 직접 연결하지 않고 **dispatch를 거칩니다**. 재배차해도 이전 배차 기록이 남습니다.
- 기사 평균 평점은 review에서 AVG로 계산합니다. 배차 정렬이 느려지면 그때 driver에 평균 컬럼을 추가합니다.
- Refresh 토큰은 수업 방식처럼 서버에 저장하지 않는(stateless) 구조로 시작합니다.
- **테이블 변경은 ERD 관리자(팀원 2)에게 요청**하고 주간 회의에서 확정합니다 (11.5).

8 API 명세

8.1 공통 규칙

- **기본 경로:** `/api` · 관리자 전용은 `/api/admin/**`, 기사 전용은 `/api/driver/**`
- **인증:** `Authorization: Bearer {accessToken}`. 401 = 로그인 필요(토큰 없음 · 만료), 403 = 권한 없음
- **목록:** 수업의 `PageRequestDTO` · `PageResponseDTO` 구조 재사용, 검색 파라미터는 `status, from, to, keyword` 로 통일
- **에러 응답:** `{ "code": "INVALID_STATUS_CHANGE", "message": "배차완료 상태에서만 상차 처리할 수 있습니다." }` + HTTP 상태코드
- **문서:** 각자 컨트롤러에 Swagger 설명을 달고, 팀원 3이 `docs/api.md` 와 Swagger 화면을 최신으로 유지합니다.

8.2 팀원 1 회원 · 인증 · 운영

메서드	URL	설명	권한
POST	`/api/auth/signup`	회원가입 (필수 약관 동의 포함, 기사는 PENDING)	전체
POST	`/api/auth/login`	로그인 → 토큰 · 역할	전체
POST	`/api/auth/refresh`	토큰 재발급	전체
GET · PUT	`/api/members/me`	내 정보 조회 · 수정	로그인
GET	`/api/terms`	가입 화면용 약관 목록	전체
GET	`/api/notices` , `/{id}`	공지 목록 · 상세	전체
GET	`/api/faqs`	FAQ 목록 (category)	전체

메서드	URL	설명	권한
POST · GET	`/api/inquiries`, `/my`	문의 등록 · 내 문의	로그인
GET	`/api/admin/dashboard`	KPI (오늘 건수 · 이번 달 매출 · 평균 평점)	ADMIN
GET	`/api/admin/stats/daily`	일별 건수 · 매출 (from, to)	ADMIN
GET · PATCH	`/api/admin/members`, `/{id}/enabled`	회원 검색 · 비활성화	ADMIN
POST · PUT · DELETE	`/api/admin/notices` · `/faqs` · `/terms`	공지 · FAQ · 약관 관리 (`/{id}`)	ADMIN
PATCH	`/api/admin/inquiries/{id}/answer`	문의 답변 (작성자에게 알림)	ADMIN

8.3 팀원 2 기사 · 차량 · 배차 · 정산

메서드	URL	설명	권한
GET	`/api/vehicle-types`	차종 목록 (신청 화면용)	전체
GET · POST · PUT	`/api/admin/vehicle-types` (`/{id}`)	차종 · 요금 관리	ADMIN
GET · POST · PUT · DELETE	`/api/admin/vehicles` (`/{id}`)	차량 관리 (DELETE = 비활성화)	ADMIN
GET · PATCH	`/api/admin/drivers`, `/{id}/status`	기사 목록 · 승인 / 중지	ADMIN
GET · POST · DELETE	`/api/driver/schedules` (`/{id}`)	내 근무 일정 (month)	DRIVER
GET	`/api/admin/dispatches/candidates`	배차 후보 (shipmentId)	ADMIN
POST	`/api/admin/dispatches`	배차 { shipmentId, driverId, vehicleId }	ADMIN
PATCH	`/api/admin/dispatches/{id}/release`	배차 해제	ADMIN
GET	`/api/driver/dispatches`	내 배차 목록 (tab = today · upcoming · done)	DRIVER
GET	`/api/driver/stats`	내 실적 · 평점 (month)	DRIVER
POST · GET	`/api/admin/settlements`	월 정산 생성 · 목록 (month)	ADMIN
PATCH	`/api/admin/settlements/{id}/pay`	지급 완료 처리	ADMIN
GET	`/api/admin/stats/drivers`	기사별 실적 순위 (month)	ADMIN

8.4 팀원 3 배송 접수 · 운임 · 결제

메서드	URL	설명	권한
GET	`/api/cargo-types`	화물 종류 목록	전체

메서드	URL	설명	권한
GET · POST · PUT	`/api/admin/cargo-types` (`/{id}`)	화물 종류 관리	ADMIN
POST	`/api/fares/estimate`	예상 운임 → 좌표 · 거리 · 추천 차종 · 운임 내역	CUSTOMER
POST	`/api/shipments`	배송 신청 → 결제대기 + 결제 준비(주문번호 · 금액)	CUSTOMER
POST	`/api/payments/confirm`	결제 승인 { paymentKey, orderId, amount } → 접수	CUSTOMER
POST	`/api/shipments/{id}/payment/retry`	결제 재시도 (새 주문번호 발급)	CUSTOMER
GET	`/api/shipments/my`	내 배송 목록 (page, status, from, to)	CUSTOMER
GET	`/api/shipments/{id}`	배송 상세 (본인 · 배정 기사 · 관리자만)	로그인
PATCH	`/api/shipments/{id}/cancel`	취소 { reason } → 결제 자동 취소(환불)	CUSTOMER · ADMIN
GET	`/api/admin/shipments`	전체 배송 검색 (status, from, to, keyword, page)	ADMIN
GET	`/api/admin/payments`	결제 · 환불 내역 (status, from, to)	ADMIN

8.5 팀원 4 배송 진행 · 실시간 · 리뷰

메서드	URL	설명	권한
PATCH	`/api/shipments/{id}/status`	상태 변경 { nextStatus, memo } — 배정된 기사만	DRIVER
GET	`/api/shipments/{id}/history`	상태 이력 (타임라인)	로그인
POST	`/api/shipments/{id}/images`	사진 업로드 (multipart: files, type)	DRIVER
GET	`/api/files/{fileName}`	사진 보기 (썸네일은 `s_` 접두어)	전체*
POST	`/api/shipments/{id}/locations`	위치 전송 { lat, lng } — 운송중만	DRIVER
GET	`/api/shipments/{id}/locations/latest`	마지막 위치 (화면 첫 진입용)	로그인
GET	`/api/notifications/subscribe`	SSE 구독 (알림 · 위치 이벤트)	로그인
GET · PATCH	`/api/notifications` , `/{id}/read`	알림 목록 · 읽음 처리	로그인
POST	`/api/shipments/{id}/review`	리뷰 작성 (배송완료 · 1회)	CUSTOMER
GET	`/api/drivers/{id}/reviews`	기사 리뷰 목록 · 평균 평점	로그인

* `<img>` 태그는 JWT 헤더를 보낼 수 없어서 파일 조회는 인증 없이 열고 파일명을 UUID로 만들어 추측할 수 없게 합니다 (수업 방식).

8.6 결제 흐름과 요청 · 응답 예시

① 신청 저장 (결제대기) → **② 토스 결제창 (테스트)** → **③ successUrl로 복귀** → **④ 서버 승인 (금액 검증)** → **⑤ 접수 + 관리자 알림**

실패하거나 창을 닫으면 failUrl 페이지에서 재시도합니다. 30분 안에 결제하지 않으면 자동 취소됩니다.

```
// ① POST /api/shipments  (요청)
{
  "originAddress": "서울특별시 강남구 OO로 12",   "originDetail": "1층 하역장",
  "destAddress":   "경기도 수원시 영통구 OO로 34", "destDetail":   "B동 창고",
  "cargoTypeId": 2,  "cargoName": "유리 진열장",  "cargoWeightKg": 600,  "vehicleTypeId": 2,
  "pickupAt": "2026-12-03T09:00:00",
  "receiverName": "김수령",  "receiverPhone": "010-0000-0000",  "memo": "엘리베이터 없음"
}
// 201 Created — 운임은 서버가 계산 (1톤 · 38.4km · 평일 오전 · 파손주의 +5,000원)
{ "id": 1024, "status": "PENDING_PAYMENT", "orderId": "SHP-1024-7f3a", "orderName": "1톤 운송 · 유리 진열장", "amount": 98500 }

// ④ POST /api/payments/confirm  (successUrl 페이지가 받은 값을 그대로 전달)
{ "paymentKey": "{토스가 넘겨준 값}", "orderId": "SHP-1024-7f3a", "amount": 98500 }
// 200 OK
{ "shipmentId": 1024, "status": "REQUESTED", "paymentStatus": "PAID" }
```

9 화면 설계

화면마다 담당자를 정했습니다. 와이어프레임은 박스 수준이면 충분하고, 각 화면 옆에 **호출하는 API**를 적어 두면 개발 순서가 분명해집니다.

구분	경로	화면	주요 구성	담당
공통	`/login`, `/signup`	로그인 · 가입	역할 선택, 필수 약관 동의 체크, 기사는 면허 · 경력 입력	1
	`/notices`, `/faq`	공지 · FAQ	목록 · 상세, FAQ 카테고리 탭	1
	`/inquiries`	1:1 문의	문의 작성, 내 문의 · 답변	1
	`/mypage`	마이페이지	내 정보 수정	1
	(레이아웃)	공통 틀	역할별 레이아웃 · 메뉴, 보호 라우트	1
	(공통 헤더)	알림	종 아이콘 + 안 읽은 개수, 드롭다운, 새 알림 토스트	4
고객	`/customer/shipments/new`	배송 신청	주소 검색 → 지도 마커, 화물 종류 · 무게 → 차종 추천, **예상 운임 카드**, [결제하기] → 토스 결제창	3
	`/customer/payments/success` · `/fail`	결제 결과	승인 요청 중 표시 → 완료 화면 / 실패 사유 · 재시도 버튼	3
	`/customer/shipments`	내 배송 목록	상태 탭, 기간 필터, 결제대기 건은 [결제하기] 버튼	3
	`/customer/shipments/:id`	배송 상세	기본 정보 · 결제 정보 · 취소 버튼(3) + 타임라인 · **실시간 기사 위치 지도** · 사진 · 리뷰 작성(4의 컴포넌트)	3 4
기사 모바일	`/driver/schedule`	근무 일정	달력에서 날짜별 근무 시간 · 휴무 등록	2
	`/driver/dispatches`	배차 목록	오늘 / 예정 / 완료 탭, 카드형 목록	2
	`/driver/dispatches/:id`	배송 처리	지도, **"다음 단계" 버튼 하나**, 사진 촬영 · 업로드, 위치 공유 상태("공유 중 · 10초 전 전송"), 수령인 전화	4
	`/driver/stats`	실적 · 평점	월 선택, 완료 건수 · 운임 합계 · 정산 상태 · 평균 평점 · 리뷰	2

구분	경로	화면	주요 구성	담당
관리자	`/admin`	대시보드	KPI 카드(오늘 접수 · 진행 · 완료 · 이번 달 매출 · 평균 평점), 일별 추이 · 차종별 차트	1
	`/admin/shipments`, `/payments`	배송 · 결제	검색 필터, 표, 상세 모달, 취소(환불) / 결제 · 환불 내역	3
	`/admin/dispatch`	배차 관리	왼쪽: 배차 대기 목록 / 오른쪽: 추천 기사(평점 · 오늘 배차 수 표시) · [배정]	2
	`/admin/drivers`	기사 관리	승인 대기 탭, 승인 · 중지, 평점	2
	`/admin/vehicle-types`, `/vehicles`	차종 · 차량	차종별 요금 표 수정 / 차량 등록 · 담당 기사 지정	2
	`/admin/cargo-types`	화물 종류	추가요금 · 취급 안내 수정	3
	`/admin/settlements`	정산	월 선택 → 정산 생성 → 지급 처리	2
	`/admin/members` 외	운영 관리	회원 · 공지 · FAQ · 약관 · 문의 답변	1

화면 공통 규칙 (UI 통일 관리: 팀원 4)

- 상태 배지 색은 6장 다이어그램 색과 맞추고, `StatusBadge` 컴포넌트 하나만 씁니다.
- 버튼 · 입력창 · 모달 · 표 · 페이지네이션은 `components/common` 의 공통 컴포넌트를 씁니다. 새로 만들기 전에 팀원 4에게 확인합니다.
- 모든 목록에 로딩 · 빈 목록 · 에러 표시, 검색 조건 · 페이지는 URL 쿼리스트링에 유지 (수업의 `useSearchParams` 방식).

10 프로젝트 폴더 구조

수업 mallapi의 계층별 패키지 구조를 그대로 따르고, **파일 단위로 담당**을 나눕니다. 같은 파일을 두 사람이 고칠 일은 공통 파일에서만 생깁니다.

10.1 백엔드 (STS) — transport-api

```
src/main/java/com/{팀명}/transport          [1]~[4] = 담당 팀원, [공통] = PR 승인 필요
├─ config/      SecurityConfig, CustomServletConfig(CORS), QuerydslConfig, SwaggerConfig [공통]
├─ security/    JWTUtil, filter/JWTCheckFilter, handler/ [1]
├─ domain/      BaseEntity [공통]
│  ├─ Member, Term, TermAgreement, Notice, Faq, Inquiry [1]
│  ├─ Driver, Vehicle, VehicleType, DriverSchedule, Dispatch, Settlement [2]
│  ├─ Shipment, CargoType, Payment [3]
│  ├─ ShipmentStatusHistory, ShipmentImage, LocationLog, Notification, Review [4]
│  └─ enums/    ShipmentStatus [3], VehicleTypeCode [2], PaymentStatus [3] ...
├─ repository/  엔티티별 JpaRepository (+ search/ 동적 검색) — 엔티티 담당자가 작성
├─ service/     도메인별 Service — 예: DispatchService [2], PaymentService [3], StatusService [4]
├─ client/      KakaoLocalClient, KakaoDirectionsClient, TossPaymentsClient [3]
├─ scheduler/   PaymentExpireScheduler [3], LocationCleanupScheduler [4]
├─ controller/  도메인별 Controller, advice/GlobalExceptionHandler [공통]
├─ dto/         요청 · 응답 DTO, PageRequestDTO, PageResponseDTO
├─ util/        CustomFileUtil [4], FareCalculator [3], GeoUtil [3]
└─ exception/   CustomException, ErrorCode (도메인별 구역) [공통]

src/main/resources/
├─ application.yml          공통 설정 [공통]
├─ application-local.yml    로컬 DB · 키, 각자 작성 (Git 제외, .example만 커밋)
└─ application-prod.yml     RDS · S3 · 운영 키 [1] (Git 제외)
```

10.2 프론트엔드 (VS Code) — transport-web

```
transport-web/
├─ .env            VITE_API_HOST, VITE_KAKAO_JS_KEY, VITE_TOSS_CLIENT_KEY (Git 제외)
└─ src/
   ├─ api/         jwtAxios [공통]
   │               memberApi, adminApi [1] · fleetApi, dispatchApi [2]
   │               shipmentApi, paymentApi [3] · trackingApi, reviewApi, notificationApi [4]
   ├─ components/  common/ [공통 · 4 관리] · map/, fare/ [3] · dispatch/ [2] · tracking/, review/ [4]
   ├─ layouts/     BasicLayout, CustomerLayout, DriverLayout, AdminLayout [1]
   ├─ pages/       auth/, board/ [1] · customer/ [3 · 4] · driver/ [2 · 4] · admin/ [화면별 담당]
   ├─ router/      root.jsx [공통] + customerRouter, driverRouter, adminRouter
   ├─ slices/      loginSlice [1]
   ├─ hooks/       useCustomLogin [1] · useNotification, useLocationShare [4] · useDebounce
   └─ util/        jwtUtil, cookieUtil, format(금액 · 날짜)
```

11 팀 협업 규칙

4명이 같은 저장소에서 일하면 코드보다 **합치는 과정**에서 시간을 더 많이 잃습니다. 1주차에 아래 규칙에 합의하고 시작하세요.

11.1 Git 브랜치 전략

`feature/{도메인}-{기능}` → PR · 리뷰 1명 → **develop (통합 · 항상 실행 가능)** → 주차 완료 시 → **main (발표 · 배포용)**

항목	규칙
main	직접 push 금지(브랜치 보호). 주차 완료 기준을 통과한 develop만 팀원 2가 병합
develop	PR로만 병합. 빌드가 깨지면 **모든 작업보다 먼저** 고칩니다
작업 브랜치	`feature/dispatch-candidate`, `fix/payment-amount-check` 처럼 도메인-기능 이름. 하루 1번 이상 develop을 받아와(pull · rebase) 충돌을 작게 유지
커밋 메시지	`feat: 배차 후보 조회 API (#12)`, `fix:` · `refactor:` · `docs:` · `test:` · `chore:` + 한글 요약 + 이슈 번호
.gitignore	`.env`, `application-local.yml`, `application-prod.yml`, 업로드 폴더, `build/`, `node_modules/` — 대신 `*.example` 파일을 커밋

11.2 PR · 코드 리뷰 규칙

- **작게:** PR 하나에 기능 하나, 300줄 안팎. 큰 기능은 "엔티티 · API" → "화면"으로 나눠 올립니다.
- **짝 리뷰:** 서로 코드를 많이 호출하는 사람끼리 짝을 짓습니다 — 1 ↔ 3, 2 ↔ 4. 공통 파일을 건드리는 PR은 그 파일 담당자도 승인합니다.
- **24시간 안에 리뷰:** 리뷰가 늦으면 다른 사람이 막힙니다. 급하면 메신저로 요청합니다.
- **PR 템플릿:** 무엇을 · 왜 / 테스트 방법(Postman 결과 · 화면 캡처) / 관련 이슈 / 공통 파일 수정 여부 / AI로 만든 부분
- **병합:** Squash merge → 작업 브랜치 삭제 → 다른 팀원은 develop을 받아옵니다.

11.3 이슈 · 회의 · 기록

무엇	방법
이슈 보드	GitHub Projects: To do / In progress / Review / Done. A ~ Z 단계의 체크리스트 항목을 이슈로 만들고 담당자 · 주차 라벨을 붙입니다
데일리 스크럼	매일 10분(메신저도 가능): 어제 한 일 · 오늘 할 일 · 막힌 것. 약속(2.4) 지연은 여기서 바로 공유
주간 회의	주 1회 30 ~ 60분: 각자 완료 기준 시연 → 다음 주 계획 → ERD · API 변경 확정 → 리스크 점검
결정 기록	`docs/decisions.md` 에 날짜 · 결정 · 이유를 한 줄씩. "왜 이렇게 했지?" 논쟁을 줄이고 발표 자료가 됩니다
트러블슈팅 노트	각자 `docs/troubleshooting/{이름}.md` 에 문제 → 원인 → 해결 기록 (Z단계 발표 소재)

11.4 공통 파일 관리

공통 파일	관리자	규칙
`SecurityConfig`, `JWTCheckFilter`	1	URL 권한 추가는 팀원 1에게 요청하거나 PR에서 승인받기
`application.yml`, `build.gradle`	1	의존성 추가는 PR 설명에 이유를 적기 (버전은 수업과 동일하게)
`ErrorCode`, `GlobalExceptionHandler`	1	ErrorCode는 도메인별 구역(회원 · 배차 · 배송 · 진행)으로 나눠 각자 자기 구역 아래에만 추가

공통 파일	관리자	규칙
`data.sql` (시드 데이터), `docs/erd.md`	2	테이블 · 초기 데이터 변경은 팀원 2가 반영
`docs/api.md`, Swagger 설정	3	API를 바꾸면 PR에 문서 수정 포함
`package.json`, `router/root.jsx`, `layouts/`	1	라이브러리 추가 · 라우트 등록은 PR로
`components/common/`, Tailwind 설정	4	공통 컴포넌트 수정 · 추가는 팀원 4 승인
`CLAUDE.md`	전원	규칙 변경은 주간 회의에서 합의 후 PR

11.5 DB · 스키마 변경 규칙

- 각자 **로컬 DB**를 같은 이름 · 버전으로 쓰고, 개발 중에는 `ddl-auto: update` 를 씁니다.
- 엔티티(테이블)를 바꾸는 PR에는 `docs/erd.md` 수정을 함께 넣고, 병합되면 메신저에 공지합니다.
- 공지를 받은 팀원은 로컬 DB를 초기화하고(`create` 로 한 번 실행) 시드 데이터를 다시 넣습니다. `update` 는 지운 컬럼을 삭제하지 않아 팀원마다 DB가 달라지기 쉽습니다.
- 시드 데이터(관리자 · 고객 · 기사 계정, 차종 · 화물 종류, 테스트 차량)는 `data.sql` 하나로 공유합니다.

12 전체 일정 (10주 · 4인 병렬)

괄호 안 알파벳은 13장의 단계입니다. 각 주의 **완료 기준**을 주간 회의에서 시연해 통과하면 develop을 main에 병합합니다.

주차	팀원 1 공통 · 인증 · 운영	팀원 2 기사 · 배차 · 정산	팀원 3 배송 · 운임 · 결제	팀원 4 진행 · 실시간 · 리뷰	완료 기준
1주	**전원 · 기획 (A ~ D):** 역할 · 협업 규칙 확정(A), 각자 도메인 요구사항 초안 → 팀원 1 취합(B), 배송 상태 확정(C), 각자 화면 목록 · 와이어프레임(D)				**요구사항 정의서 · 상태 다이어그램 · 화면 목록**
2주	저장소 · 브랜치 보호 · PR 템플릿 · CLAUDE.md (H), 공통 골격 1차 (I)	ERD 통합 (E) — 전원 설계 취합, 시드 데이터 초안	API 명세 통합 · Swagger (F), 카카오 앱 · 토스 테스트 키 (G)	상태 · 알림 · 위치 설계, UI 규칙(색 · 배지 · 공통 컴포넌트)	**ERD · API 확정, 전원 빈 프로젝트 실행**
3주	회원가입 · 약관 · JWT (J), 레이아웃 · 라우팅 · 로그인 (K)	Driver · Vehicle · VehicleType 엔티티 · 관리자 API (L)	Shipment · CargoType 엔티티 먼저 공유, 거리 조회 · 운임 계산 · 테스트 (M)	알림 엔티티 · SSE 구독, 상태 이력 엔티티, FileStorage (P · R 기반)	**역할별 로그인, 운임 테스트 통과**
4주	공지 · FAQ · 문의 (V), 마이페이지, 관리자 회원 · 약관	차종 · 차량 · 기사 관리 화면, 기사 근무 일정 (L)	배송 신청 화면(주소 · 지도 · 예상 운임), 신청 API → 결제대기 (N)	상태 변경 API · 이력 · 타임라인 컴포넌트 (P)	**지도 보며 신청 → 결제대기 저장**
5주	공통 컴포넌트 · 권한 정리, 통합 지원, 대시보드 쿼리 설계 (T)	배차 후보 · 배정 · 해제, 기사 배차 목록 (O)	토스 테스트 결제 승인 · 실패 · 재시도, 결제대기 자동 취소 (N)	기사 배송 처리 화면, 사진 업로드 (Q)	**결제 → 접수 → 배차 동작**
6주	통합 이슈 관리, 권한 · 보안 점검	재배차 · 근무 일정 조건 검증	취소 · 환불, 내 배송 목록 · 상세	배차 · 상태 변경 알림 연결 (R)	**MVP: 신청 → 완료 통합 (전원 테스트)**
7주	관리자 대시보드 · 통계 (T)	월 정산 (U), 기사 실적 화면	관리자 통합 검색 · 결제 내역 (S), 화물 종류 관리	실시간 위치 공유 (R) — HTTPS 터널로 휴대폰 테스트	**고객 지도에서 기사 위치 이동**
8주	1차 배포 · HTTPS (Y)	배차 정렬에 평점 반영, 정산 검증	시연용 더미 데이터, N+1 점검	리뷰 · 평점 (Q), 평점 조회 메서드 제공	**배포 URL에서 전체 흐름 동작**
9주	**전원 · 마무리 (W ~ Y):** 도메인별 테스트(W), UI 통일(X · 팀원 4 주도), 버그 수정, 최종 배포(Y · 팀원 1)				**배포 URL에서 시연 시나리오 통과**
10주	**전원 · 발표 (Z):** 도메인별 README · 트러블슈팅, 발표 자료 각자 파트, 시연 영상 (취합 팀원 1). 남는 시간은 예비 버퍼				**제출물 완성**

> **일정이 밀릴 때**
> 6주차 MVP 통합이 최우선입니다. 확장 기능이 늦어지면 **공지 · FAQ → 정산 → 리뷰 → 통계 → 위치 추적** 순서로 뒤로 미루고 10주차 버퍼를 씁니다. 지도 · 운임 · 결제 · 알림은 시연의 중심이므로 끝까지 지킵니다. 한 사람이 밀리면 일정에 여유가 있는 사람이 그 도메인의 화면 일부를 나눠 맡습니다.

12.1 주간 진행 방식

- **월요일:** 이번 주 이슈 확인 → 각자 Claude Code에 단계 단위로 계획을 먼저 받습니다.
- **주중:** 기능 하나씩 "엔티티 · API → Postman 확인 → 화면 연결 → PR → 리뷰 · 병합" 순서로 진행합니다.
- **금요일(주간 회의):** 완료 기준을 develop 기준으로 시연하고, 통과하면 main에 병합합니다.

13 A ~ Z 단계별 상세 작업

26단계를 6개 페이즈로 나눴습니다. 4명이 동시에 진행하므로 **알파벳 순서는 의존 순서이지 시간 순서가 아닙니다.** 주차는 12장 일정표를 따르고, 각 단계의 담당 표시를 확인하세요.

Phase 1 · 기획 A ~ D · 1주차 · 전원

4명이 함께 무엇을 만들지 확정합니다. 여기서 정한 내용이 이후 모든 단계와 역할 분담의 기준이 됩니다.

A 팀 구성 · 목표 · 범위 확정

전원 · 1주차

목표	누가, 무엇을, 어디까지 만들지 정하고 협업 방식을 합의합니다.
할 일	☐ 2장 역할 분담표에 이름 채우기 (공통 책임 포함) ☐ 포함 · 제외 범위 확정 (1.5), 서비스 한 줄 소개 ☐ 11장 협업 규칙 합의: 브랜치 · PR · 리뷰 짝 · 회의 시간 · 메신저 채널 ☐ 제출일 · 발표일 확인 후 12장 일정 조정
산출물	역할 분담표(이름 포함), 협업 규칙 합의, 프로젝트 개요 (README 첫 부분)

B 요구사항 정의

전원 · 취합 1 · 1주차

목표	만들 기능을 빠짐없이 목록으로 만들고 번호 · 담당자를 붙입니다.
할 일	☐ 5장 표를 기준으로 각자 자기 도메인 요구사항 초안 작성 → 팀원 1이 취합 ☐ 기능마다 “누가, 무엇을 입력하고, 결과로 무엇을 보는지” 한 줄씩 ☐ 예외 상황 정리: 결제 실패 · 이탈, 취소 · 환불, 재배차, 사진 누락, 위치 권한 거부, 승인 안 된 기사
산출물	요구사항 정의서 (`docs/requirements.md`)
Claude Code	“docs/requirements.md를 보고 빠진 기능이나 예외 상황을 찾아 목록으로 알려줘. 파일은 아직 고치지 마.”

C 배송 상태 · 업무 흐름 확정

전원 · 1주차

목표	4명이 함께 쓰는 핵심 규칙인 상태 전이를 확정합니다.
할 일	☐ 상태 8개(결제대기 포함)와 의미 확정 (6.1) ☐ 전이마다 바꾸는 사람 · 조건 · 함께 처리할 일 · 담당 확정 (6.3) ☐ 고객 · 관리자 · 기사 역할을 나눠 맡아 처음부터 끝까지 말로 시연해 보기
산출물	상태 정의표, 상태 다이어그램 (`docs/status.md`)
주의	상태를 나중에 바꾸면 4명의 코드가 모두 바뀝니다. 이 단계에서 확정하세요.

D 화면 목록 · 와이어프레임

전원 · 1주차

목표	만들 화면 전체와 담당자, 각 화면이 부르는 API를 미리 연결합니다.
할 일	☐ 9장 화면 목록 확정, 각자 자기 화면 와이어프레임 (손그림 · Figma) ☐ 화면 옆에 호출할 API 메모 (예: 배송 신청 → `/api/fares/estimate`, `/api/shipments`) ☐ 여러 사람이 함께 쓰는 화면(배송 상세)은 영역별 담당 표시 ☐ 기사 화면은 휴대폰 크기 기준
산출물	화면 목록(담당 표시), 와이어프레임 이미지

Phase 2 · 설계 · 준비 E ~ H · 2주차

설계 문서를 완성하고, 외부 서비스 설정과 저장소 · 개발 환경을 끝냅니다.

E ERD 설계

전원 · 취합 2 · 2주차

목표	20개 테이블 구조를 확정합니다.
할 일	☐ 각자 자기 도메인 테이블 설계 (7장) → 팀원 2가 ERDCloud에 통합 ☐ 도메인 사이 외래키 확인: shipment ↔ dispatch · payment · review · location_log ☐ PK · FK · UNIQUE · NULL, enum 값 목록 확정 ☐ 샘플 데이터를 직접 써 보며 빠진 컬럼 확인 → 시드 데이터(`data.sql`) 초안
산출물	ERD 이미지, 테이블 정의서 (`docs/erd.md`)
Claude Code	“docs/erd.md를 JPA 엔티티로 옮길 때 문제가 될 부분(연관관계 방향, N+1 가능성, 빠진 인덱스·UNIQUE)을 검토해 줘.”

F API 명세 작성

전원 · 취합 3 · 2주차

목표	4명이 공유하는 약속 문서를 만듭니다.
할 일	☐ 각자 8장 자기 표를 기준으로 URL · 권한 · 요청 / 응답 JSON 예시 작성 → 팀원 3 취합 ☐ 에러 코드 목록: `INVALID_STATUS_CHANGE`, `NOT_ASSIGNED_DRIVER`, `PHOTO_REQUIRED`, `ALREADY_DISPATCHED`, `PAYMENT_AMOUNT_MISMATCH` 등 ☐ Swagger(springdoc) 설정, Postman 팀 워크스페이스 만들기 ☐ 2.4 도메인 간 약속 메서드의 이름 · 파라미터 확정
산출물	API 명세 (`docs/api.md`), 약속 메서드 목록

G 외부 서비스 준비

팀원 3 · 팀원 4 · 2주차

목표	코드 밖에서 사람이 직접 해야 하는 설정을 미리 끝냅니다.
할 일	☐ (3) 카카오 디벨로퍼스에서 **팀 앱 하나** 생성 → JavaScript SDK 도메인에 `http://localhost:5173` 등록 → 카카오맵 API 활성화(무료 쿼터 대상인지 확인) → 팀원 초대 또는 키 공유 ☐ (3) 토스페이먼츠 개발자센터 가입 → 테스트용 API 개별 연동 키(클라이언트 · 시크릿) 확인, 시크릿 키는 서버 설정 파일에만 ☐ (3) 카카오모빌리티 길찾기 API 사용 조건 · 무료 쿼터 확인 ☐ (4) HTTPS 터널 도구 설치 · 휴대폰에서 위치 권한 요청이 뜨는지 확인 ☐ (전원) 키는 메신저 공지방 대신 비공개 메모로 공유, GitHub에 절대 올리지 않기
산출물	설정된 카카오 앱 · 토스 테스트 키, 키 공유 방법 합의
주의	카카오 API 좌표는 `x = 경도`, `y = 위도`입니다. 순서를 바꾸면 바다 한가운데가 나옵니다.

H 개발 환경 · 저장소 · 협업 세팅

팀원 1 · Git 2 · 2주차

목표	4명 모두 같은 프로젝트를 받아 실행하고, 규칙대로 PR을 올릴 수 있는 상태를 만듭니다.
할 일	☐ (1) Spring Initializr로 백엔드 생성 (수업과 같은 버전), Vite로 React 생성 → Tailwind · Router · Redux Toolkit · axios ☐ (2) GitHub 저장소 · 팀원 초대, main · develop 브랜치 보호, PR 템플릿, Projects 이슈 보드 ☐ (1) CLAUDE.md와 `docs/` 폴더, `application-local.yml.example` · `.env.example` ☐ (전원) clone → 로컬 DB 생성 → 백엔드 · 프론트 실행 → 프론트(5173)에서 백엔드(8080) 호출 성공
산출물	4명 모두 실행되는 빈 프로젝트, 첫 PR 병합
Claude Code	"CLAUDE.md를 읽고 백엔드 기본 패키지 구조와 BaseEntity, CORS 설정을 만들어줘. 만들기 전에 파일 목록부터 보여줘."

Phase 3 · 공통 기반 I ~ K · 2 ~ 3주차 · 팀원 1 중심

모두가 그 위에서 작업하는 공통 코드와 로그인 · 권한입니다. 다른 팀원은 이 기간에 자기 엔티티 · API를 시작합니다.

I 백엔드 공통 골격

팀원 1 · 2주차

목표	모든 도메인이 쓰는 기반 코드를 2주차 말까지 develop에 올립니다.
할 일	☐ `BaseEntity` (JPA Auditing), `PageRequestDTO` · `PageResponseDTO` (수업 코드) ☐ `CustomException` + 도메인별 구역으로 나눈 `ErrorCode` + `GlobalExceptionHandler` ☐ CORS, 시간대 통일, Swagger 기본 설정, 개발용 임시 보안 설정(전체 허용) ☐ 시드 데이터 연결 (`data.sql` — 내용은 팀원 2가 관리)
산출물	공통 코드 (2.4 약속 1번)

J 회원 · 인증 · 권한 · 약관

팀원 1 | 3주차

목표	역할별로 안전하게 로그인하고 API 접근을 나눕니다.
할 일	☐ Member · Term · TermAgreement 엔티티, 회원가입(필수 약관 동의 검사 · 이력 저장, BCrypt) ☐ 기사로 가입하면 Driver(PENDING) 생성 — 팀원 2의 `DriverService` 메서드 호출 ☐ 로그인 → Access(예: 30분) · Refresh(예: 1일) 발급, 재발급 API, `JWTCheckFilter` ☐ URL 권한: `/api/admin/**` → ADMIN, `/api/driver/**` → DRIVER, 나머지 인증 필요 (+ `@PreAuthorize`) ☐ 로그인 사용자 꺼내는 방법을 팀에 공유 (2.4 약속 2번)
산출물	가입 · 로그인 · 재발급 API, 역할별 권한 테스트
주의	임시 "전체 허용" 설정을 실제 권한으로 바꾸는 날을 미리 공지하세요. 다른 팀원의 Postman 테스트가 갑자기 401이 됩니다.

K 프론트 공통 골격

팀원 1 | UI 4 | 3주차

목표	역할별 레이아웃 · 라우팅 · 로그인 유지와 공통 컴포넌트를 만듭니다.
할 일	☐ (1) 레이아웃 4종, 보호 라우트, `loginSlice` + 쿠키, 새로고침 후 로그인 유지 ☐ (1) axios 인터셉터: Access 첨부, 만료 시 Refresh로 재발급 후 재요청, 실패 시 로그아웃 ☐ (1) 로그인 · 회원가입(약관 동의) 화면 ☐ (4) 공통 컴포넌트: Button · Input · Modal · Table · Pagination · Loading · StatusBadge, 색 규칙
산출물	역할별로 로그인하면 각자 첫 화면으로 이동
Claude Code	"수업의 jwtUtil 인터셉터 구조를 그대로 쓰되, 재발급이 실패하면 로그아웃하고 /login으로 보내도록 만들어줘."

Phase 4 · 핵심 기능 L ~ P · 3 ~ 6주차

"신청 → 결제 → 배차 → 운송 → 완료" 흐름을 도메인별로 동시에 만들고, 6주차에 통합합니다.

L 차종 · 차량 · 기사 · 근무 일정 관리

팀원 2 | 3 ~ 4주차

목표	배차의 재료인 차종 · 차량 · 기사 · 일정을 관리합니다.
할 일	☐ Driver · Vehicle · VehicleType(요금 컬럼 포함) 엔티티 → 팀원 3에게 VehicleType 먼저 공유 ☐ 관리자: 기사 승인 · 중지, 차종 · 요금 표 수정, 차량 등록 · 비활성화 · 담당 기사 지정 ☐ DriverSchedule: 기사가 달력에서 날짜별 근무 시간 · 휴무 등록 (기사+날짜 UNIQUE) ☐ 화면: `/admin/drivers`, `/admin/vehicle-types`, `/admin/vehicles`, `/driver/schedule`
산출물	관리자 CRUD, 기사 근무 일정 등록
Claude Code	"Vehicle 엔티티부터 Controller까지 수업 mallapi의 Product 구조를 참고해서 만들고, 파일마다 하는 일을 설명해줘."

M 화물 종류 · 거리 조회 · 운임 계산

팀원 3 | 3주차

목표 거리 · 차종 · 화물 종류 · 시간 · 무게로 운임을 자동 계산합니다. (규칙은 바로 아래)

할 일

- [ ] Shipment · CargoType 엔티티 · `ShipmentStatus` 를 3주차 중에 먼저 공유 (2.4)
- [ ] `KakaoLocalClient` (주소 → 좌표), `KakaoDirectionsClient` (도로 거리 · 소요시간), 실패 시 하버사인 직선거리 × 1.3
- [ ] `FareCalculator` + 단위 테스트 (아래 예시 세 개가 그대로 나오는지), `POST /api/fares/estimate`
- [ ] 같은 출발 · 도착 구간 결과 재사용으로 무료 쿼터 절약

산출물 운임 계산 API, 단위 테스트 통과

M단계 상세 — 운임 계산 규칙 (예시 값이며 실제 시세가 아닙니다)

차종 (vehicle_type)	최대 적재	기본요금 (10km 포함)	초과 1km당	무게 기준 자동 추천
`SMALL` 소형 (다마스 · 라보)	500kg	30,000원	1,000원	500kg 이하
`TON1` 1톤	1,000kg	50,000원	1,500원	500kg 초과 ~ 1,000kg
`TON2_5` 2.5톤	2,500kg	90,000원	2,200원	1,000kg 초과 ~ 2,500kg
`TON5` 5톤	5,000kg	150,000원	3,000원	2,500kg 초과 ~ 5,000kg (넘으면 문의 안내)

화물 종류 (cargo_type)	추가요금	취급 안내 (기사 화면에 표시)
일반	0원	—
파손주의	5,000원	완충재 포장 확인, 적재 시 위에 다른 짐 금지
가구 · 가전	10,000원	상하차 시 고객과 함께 확인

계산식: 운임 = (기본요금 + (거리를 1km 단위로 올림 − 10km) × km당 요금 + 화물 추가요금) → 할증률을 **합산해 한 번에** 적용 → 100원 미만 버림

할증: 야간(상차 22:00 ~ 06:00) +20% · 주말(토 · 일) +10% · 고중량(차종 최대 적재량의 80% 초과) +10%

예시	조건	계산	운임
① 기본	1톤, 38.4km, 600kg, 일반, 2026-12-03(목) 09:00	50,000 + (39 − 10) × 1,500 = 93,500, 할증 없음	**93,500원**
② 할증	1톤, 38.4km, 900kg, 일반, 2026-12-05(토) 22:30	93,500 × (1 + 야간 20% + 주말 10% + 고중량 10%) = 93,500 × 1.4	**130,900원**
③ 화물	1톤, 38.4km, 600kg, 파손주의, 2026-12-03(목) 09:00	93,500 + 5,000 = 98,500, 할증 없음	**98,500원**

```
public int calculate(VehicleType type, CargoType cargo,
                     double distanceKm, int weightKg, LocalDateTime pickupAt) {
    int overKm = Math.max(0, (int) Math.ceil(distanceKm) - type.getBaseDistanceKm());
    int fare = type.getBaseFare() + overKm * type.getPerKmFare() + cargo.getExtraFee();

    int rate = 0;                                              // 할증률 합산(%)
    int hour = pickupAt.getHour();
    if (hour >= 22 || hour < 6) rate += type.getNightRate();
    DayOfWeek day = pickupAt.getDayOfWeek();
    if (day == DayOfWeek.SATURDAY || day == DayOfWeek.SUNDAY) rate += type.getWeekendRate();
    if (weightKg * 100 > type.getMaxLoadKg() * 80) rate += type.getHeavyRate();   // 80% 초과

    int total = fare + fare * rate / 100;
    return total / 100 * 100;                                  // 100원 미만 버림
}
```

외부 API 참고: 카카오 로컬 주소 검색은 `GET https://dapi.kakao.com/v2/local/search/address.json?query=주소`에 `Authorization: KakaoAK {REST_API_KEY}` 헤더를 붙여 호출합니다. 길찾기 API의 요청 형식 · 무료 쿼터는 카카오모빌리티 디벨로퍼스 문서에서 확인하세요.

N 배송 신청 · 지도 · 결제

팀원 3 | 4 ~ 5주차

목표

고객이 지도를 보며 신청하고, 테스트 결제까지 마쳐 '접수' 상태를 만듭니다.

할 일

- [ ] `POST /api/shipments`: 서버에서 좌표 · 거리 · 운임 재계산 → 배송 PENDING_PAYMENT + 결제 READY(주문번호 · 금액) 저장 → 이력
- [ ] 신청 화면: 우편번호 팝업 → 지도 마커(JS SDK `services` Geocoder), 화물 종류 · 무게 → 차종 추천, 0.5초 디바운스로 예상 운임 갱신
- [ ] 토스 결제창 열기 → successUrl 페이지에서 `/api/payments/confirm` 호출 → **저장된 금액과 비교** → 토스 승인 → PAID · 접수 · 관리자 알림
- [ ] failUrl 페이지 · 재시도(새 주문번호), `@Scheduled`로 30분 지난 결제대기 자동 취소
- [ ] 승인 성공 후 DB 저장이 실패하면 결제 취소 API로 되돌리기
- [ ] local 프로필 전용 "결제 건너뛰기" API로 팀원들이 결제 없이 테스트할 수 있게 (2.4)

산출물

신청 → 테스트 결제 → 접수, 내 배송 목록에서 확인

Claude Code

"결제 승인 API를 만들어줘. orderId로 저장된 금액과 요청 금액이 다르면 PAYMENT_AMOUNT_MISMATCH로 거절하고, 토스 승인은 TossPaymentsClient로 호출. 시크릿 키는 application-local.yml에서 읽기."

O 배차

팀원 2 | 5 ~ 6주차 (+8주차 평점)

목표	조건에 맞는 기사 · 차량을 추천하고 배정합니다.
할 일	☐ 후보 조건: 기사 ACTIVE + **상차 일시가 근무 일정 안** + 차량 AVAILABLE + 최대 적재량 ≥ 무게 + 요청 차종 이상 + 당일 배차 수 < 한도(기본 3건) ☐ 정렬: 당일 배차 수 적은 순 → 적재량 여유 적은 순 → (8주차) 평균 평점 높은 순 ☐ 배정: Dispatch 생성 + ASSIGNED + 이력 + 알림 / 해제: active = false + REQUESTED ☐ `getActiveDriver(shipmentId)` 를 팀원 4에게 제공 (2.4), 기사 배차 목록 화면
산출물	관리자가 배정하면 기사 목록에 표시
주의	관리자 두 명이 같은 배송을 동시에 배정하지 못하도록 Shipment의 `@Version` 으로 막고, 충돌하면 "이미 배차된 건입니다"로 안내합니다.

P 상태 변경 · 이력

팀원 4 | 3 ~ 4주차

목표	기사가 단계별로 상태를 바꾸고, 모든 변경이 기록됩니다.
할 일	☐ 이력 저장 메서드를 먼저 만들어 팀원 2 · 3이 호출할 수 있게 공유 ☐ `PATCH /api/shipments/{id}/status` : 배정 기사 본인 확인 → `canChangeTo` → 변경 → 이력 ☐ 타임라인 컴포넌트(고객 상세 화면에 들어감) ☐ 기사 처리 화면: 현재 상태에 맞는 "다음 단계" 버튼 하나만
산출물	6주차 통합 시 접수 → 완료 전체 흐름 동작 (**MVP**)
Claude Code	"StatusService.changeStatus에 대해 정상 흐름, 잘못된 전이(REQUESTED → COMPLETED), 다른 기사의 요청을 검사하는 JUnit 테스트를 작성하고 실행 결과를 보여줘."

Phase 5 · 확장 기능 Q ~ V · 4 ~ 8주차

MVP 위에 사진 · 리뷰 · 실시간 알림 · 위치 · 검색 · 통계 · 정산 · 고객센터를 얹습니다.

Q 사진 증빙 · 리뷰

팀원 4 | 5주차 · 8주차

목표	상차 · 인수증 사진으로 증빙하고, 완료 후 고객이 기사를 평가합니다.
할 일	☐ (5주) ShipmentImage, `CustomFileUtil` 재사용, 이미지 확장자만 허용, `spring.servlet.multipart.max-file-size` 상향 (기본 파일당 1MB) ☐ (5주) 상차완료 전 PICKUP · 배송완료 전 RECEIPT 사진 서버 검사, `<input type="file" accept="image/*" capture="environment">` ☐ (8주) Review: 배송완료 건만, 배송당 1회, 별점 1 ~ 5 · 후기 → 기사 평균 평점 조회 메서드를 팀원 2에게 제공 ☐ 파일 저장은 `FileStorage` 인터페이스로 분리 (로컬 / S3)
산출물	사진 첨부 후 상태 변경, 리뷰 작성 · 기사 평점 표시

R 실시간 알림 · 기사 위치 추적 (SSE)

팀원 4 · 3 · 6 · 7주차

목표	상태가 바뀌면 바로 알림이 뜨고, 운송중에는 고객 지도에서 기사가 움직입니다.
할 일	☐ (3주) 회원별 `SseEmitter` 보관 · 완료 · 타임아웃 시 제거, heartbeat(예: 30초), `NotificationService.send` 빈 구현 공유 ☐ (6주) 알림 저장 + 접속 중이면 즉시 전송, `@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)` 로 커밋 후에만 발송 ☐ (7주) 기사 화면: 운송중일 때만 `watchPosition` → 30초에 한 번 `POST /locations` → 도착 처리 시 `clearWatch` ☐ (7주) 서버: 배정 기사 · 운송중인지 확인 → 저장 → 고객에게 SSE `location` 이벤트 → 고객 지도 마커 이동, "마지막 위치 n초 전" 표시 ☐ 위치 권한 거부 · 신호 없음 처리, 30일 지난 위치 삭제 스케줄러
산출물	기사가 상태를 바꾸면 고객 화면에 알림, 고객 지도에서 기사 위치 이동
Claude Code	"기사 배송 처리 화면에 useLocationShare 훅을 붙여줘. IN_TRANSIT일 때만 watchPosition으로 위치를 받아 30초에 한 번 전송하고, 상태가 바뀌거나 화면을 나가면 clearWatch로 멈춰."
주의	위치는 HTTPS · localhost에서만 받을 수 있습니다(3장 메모 ⑤). 기본 `EventSource` 는 헤더를 못 보내므로 `event-source-polyfill` 을 쓰고, Nginx에서 SSE 경로는 `proxy_buffering off` 와 긴 `proxy_read_timeout` 이 필요합니다. 휴대폰 화면이 꺼지면 브라우저가 위치 전송을 멈출 수 있습니다.

S 목록 · 검색 · 결제 내역

팀원 3 · 전원 점검 · 7주차

목표	데이터가 쌓여도 원하는 건을 빨리 찾게 합니다.
할 일	☐ (3) 관리자 배송 통합 검색: 상태 · 기간 · 고객명 · 기사명 · 키워드 (조건이 많으면 QueryDSL) ☐ (3) 관리자 결제 · 환불 내역, 화물 종류 관리 화면 ☐ (전원) 자기 목록 화면의 N+1 쿼리 확인 → fetch join 또는 `@EntityGraph`, 정렬 · 페이징 · URL 쿼리스트링 유지
산출물	정확한 검색 결과, 목록 1회 조회당 쿼리 수 확인

T 관리자 대시보드 · 통계

팀원 1 · 5주차 설계 · 7주차

목표	운영 현황을 한눈에 보여 줍니다.
할 일	☐ KPI: 오늘 접수 · 진행 · 완료, 이번 달 매출(결제 PAID − 환불), 평균 평점 ☐ 일별 건수 · 매출 선 차트, 차종별 막대 차트 — 집계는 DB 쿼리, 프론트는 그리기만 ☐ 다른 도메인 데이터는 각 담당자의 조회 메서드로 받기 (직접 Repository 호출 금지)
산출물	`/admin` 대시보드

U 월 정산

팀원 2 · 7주차

목표 월 단위로 기사별 지급액을 계산합니다.

할 일
- [ ] 해당 월 COMPLETED 건(환불 제외)을 기사별 합산 → 수수료 차감 → 지급액
- [ ] 이미 만든 월은 중복 생성 막기 (PENDING만 재계산), 지급 완료 처리
- [ ] 기사 실적 화면 금액과 일치하는지 확인

산출물 정산 목록 · 지급 처리 (예: 완료 42건, 운임 합계 4,200,000원, 수수료 10% → 지급액 3,780,000원)

V 공지 · FAQ · 1:1 문의

팀원 1 · 4주차

목표 운영에 필요한 고객센터 기능을 만듭니다.

할 일
- [ ] 공지 · FAQ CRUD(관리자) · 목록(전체) — 수업의 기본 CRUD · 페이징 구조 응용
- [ ] 1:1 문의 등록, 관리자 답변 → 작성자에게 알림 (팀원 4의 `send` 호출)
- [ ] 약관 관리(버전 · 필수 여부)

산출물 공지 · FAQ · 문의 · 약관 관리 동작

Phase 6 · 마무리 W ~ Z · 8 ~ 10주차

테스트 · 다듬기 · 배포 · 발표 준비로 "돌아가는 코드"를 "보여줄 수 있는 서비스"로 만듭니다.

W 테스트

전원 · 9주차

목표 시연 중에 터지지 않게 합니다.

할 일
- [ ] 각자 단위 테스트: 운임(예시 3개) · 결제 금액 검증(3), 상태 전이(4), 배차 조건 · 정산(2), 권한(1)
- [ ] 통합 테스트: 신청 → 결제 → 배차 → 완료 → 리뷰 (`@SpringBootTest`)
- [ ] 결제 시나리오: 성공 · 실패 · 창 닫기 · 30분 만료 · 취소 환불
- [ ] 권한: 다른 고객 배송 조회, 다른 기사 상태 변경 · 위치 전송 → 403
- [ ] 사람 테스트: 고객 · 관리자 · 기사(실제 휴대폰) 역할을 나눠 16장 시나리오 그대로

산출물 테스트 통과, 버그 이슈와 수정 기록

X UI 다듬기 · 예외 처리

전원 · 주도 4 · 9주차

목표 4명이 만든 화면이 한 사람이 만든 것처럼 보이게 합니다.

할 일
- [ ] 팀원 4가 전체 화면 점검표 작성 → 각자 자기 화면 수정
- [ ] 로딩 · 빈 목록 · 에러, 금액 · 날짜 형식, 상태 배지 색 통일
- [ ] 기사 화면을 실제 휴대폰으로 확인, 에러 코드를 이해되는 문장으로

산출물 화면 점검표 완료

Y 배포 (AWS · HTTPS)

팀원 1 | 8 ~ 9주차

목표

HTTPS URL로 접속할 수 있는 서비스를 8주차에 먼저 띄웁니다.

할 일

- [ ] AWS 계정(3장 메모 ③) → Budgets 예산 알림, RDS · S3 생성, `FileStorage`를 S3로 교체
- [ ] EC2: Spring Boot jar를 prod 프로필로 실행, DB · 카카오 · 토스 키는 환경변수
- [ ] React 빌드 → Nginx 정적 서빙 + `/api` 프록시 (SSE 경로 버퍼링 끔)
- [ ] **도메인 연결 + HTTPS(Let's Encrypt) 필수** — 위치 추적 때문
- [ ] 카카오 앱에 배포 도메인 추가, 토스 successUrl · failUrl을 배포 주소로, CORS 허용 주소 변경

산출물

배포 URL, 배포 절차 문서

주의

DB 포트(3306)를 전체 공개하지 마세요. 발표가 끝나면 쓰지 않는 리소스를 중지 · 삭제해 요금을 막으세요.

Z 문서화 · 발표 준비

전원 | 취합 1 | 10주차

목표

4명이 각자 자기 도메인을 자신 있게 설명합니다.

할 일

- [ ] README: 소개, 팀 · 역할, 기술 스택, 구성도, ERD, 주요 기능 GIF, API 문서 링크, 실행 방법
- [ ] 각자 트러블슈팅 2개 이상 (문제 → 원인 → 해결)
- [ ] 발표 분담: 팀원 1(문제 정의 · 구조 · 보안 · 배포), 팀원 3(신청 · 운임 · 결제), 팀원 2(배차 · 정산), 팀원 4(실시간 · 리뷰), 협업 방식 · 회고(전원)
- [ ] 16장 시나리오대로 시연 영상 녹화 (실시간 시연이 실패할 때 대비)
- [ ] 예상 질문: 왜 SSE인가? 결제 금액 위조는 어떻게 막았나? 동시 배차는? 위치 개인정보는?

산출물

README, 시연 영상, 발표 자료

14 Claude Code 활용 가이드

14.1 팀 공용 CLAUDE.md 템플릿

저장소 루트에 커밋해 두면 Claude Code가 작업할 때마다 읽습니다. 팀원 모두 같은 규칙으로 코드를 받게 됩니다.

```
# 운송서비스 운용 사이트 (transport) — 4인 팀

## 기술 스택
- Backend: Spring Boot 3.x (수업 mallapi와 같은 버전), Java 17, Gradle, JPA, Spring Security + JWT,
MariaDB
- Frontend: React + Vite, React Router, Redux Toolkit, axios, Tailwind CSS
- 외부: 카카오맵 JS SDK, 카카오 로컬 · 길찾기 API, 토스페이먼츠(테스트 키), SSE, Geolocation API

## 구조 규칙
- 백엔드 패키지: config, security, domain, repository, service, controller, dto, client, scheduler, util,
exception
- 수업 mallapi 코드 스타일을 따른다 (DTO 변환, PageRequestDTO/PageResponseDTO, CustomFileUtil)
- 연관관계는 @ManyToOne(fetch = LAZY) 기본, setter 대신 change~ 메서드

## 도메인 규칙 (자세한 내용은 docs/)
- 배송 상태: PENDING_PAYMENT → REQUESTED → ASSIGNED → PICKED_UP → IN_TRANSIT → ARRIVED → COMPLETED, 취소
CANCELED
- 상태 변경은 ShipmentStatus.canChangeTo()로만 검사하고, 변경마다 이력을 저장한다
- 운임은 서버의 FareCalculator로만 계산하고, 결제 승인 전 저장된 금액과 비교한다

## 팀 규칙
- 담당: 팀원1 공통·인증·운영 / 팀원2 기사·차량·배차·정산 / 팀원3 배송·운임·결제 / 팀원4 진행·실시간·리뷰
- 내 도메인 파일만 수정한다. 공통 파일(SecurityConfig, application*.yml, build.gradle, ErrorCode,
  router/root.jsx, package.json, components/common)은 수정하지 말고 변경안을 제안만 한다
- 다른 도메인은 docs/api.md에 정해진 Service 메서드로만 호출한다

## 작업 방식
- 코드를 바꾸기 전에 바꿀 파일 목록과 계획을 먼저 보여준다
- 한 번에 한 단계(A~Z)만 진행하고, 끝나면 실행 · 테스트 결과를 보여준다
- API 키 · 시크릿 키 · 비밀번호를 코드에 직접 쓰지 않는다 (.env, application-local.yml)
```

대화를 시작할 때 “나는 팀원 3(배송 · 운임 · 결제) 담당이야”처럼 자기 역할을 먼저 알려 주세요.

14.2 docs 폴더

`docs/requirements.md` · `status.md` · `erd.md` · `api.md` · `decisions.md`에 이 계획서의 5 ~ 8장과 회의 결정을 옮겨 두세요. “docs/api.md대로 만들어줘”라고 하면 매번 길게 설명할 필요가 없습니다.

14.3 요청하는 방법

이렇게 말하면 결과가 흔들립니다	이렇게 말하면 원하는 결과가 나옵니다
“배송 기능 만들어줘”	“Shipment 엔티티와 `POST /api/shipments`를 만들어줘. 요청 DTO는 docs/api.md 예시대로, 운임은 FareCalculator로 서버에서 다시 계산.”
“에러 나, 고쳐줘”	“로그인 후 `/api/shipments/my`를 호출하면 403이 나. 로그를 붙일게. 고치기 전에 원인부터 설명해줘. SecurityConfig는 공통 파일이니 수정안만 보여줘.”
“전부 다 만들어줘”	“N단계 중 결제 승인 부분만 진행해줘. 끝나면 테스트 실행 결과까지 보여줘.”
“PR 써줘”	“이 브랜치에서 바뀐 내용을 우리 PR 템플릿(무엇을 · 왜 / 테스트 방법 / 공통 파일 수정 여부)에 맞춰 정리해줘.”

이렇게 말하면 결과가 흔들립니다	이렇게 말하면 원하는 결과가 나옵니다
“리뷰해줘”	“팀원 PR의 diff야. 권한 검사 누락, 상태 전이 규칙 위반, N+1 가능성을 중심으로 찾아줘.” → 최종 판단은 사람이

14.4 기능 하나당 작업 루틴

① 계획 받기 → ② 한 단계 구현 → ③ 직접 확인 → ④ 설명 듣기 → ⑤ 수업 코드 비교 → ⑥ PR · 리뷰

- **③ 직접 확인:** AI가 “완료”라고 해도 직접 실행해서 Postman과 브라우저로 확인합니다.
- **④ 설명 듣기:** “방금 만든 코드의 요청 흐름을 Controller부터 순서대로 설명해줘” → 리뷰어 질문과 발표에 그대로 쓰입니다.
- **⑥ PR:** 자기가 설명할 수 있는 코드만 올립니다. AI로 만든 부분은 PR에 표시해 리뷰어가 더 꼼꼼히 봅니다.
- **대화 관리:** 단계가 바뀌면 새 대화로 시작하고(`/clear`), 필요한 맥락은 CLAUDE.md와 docs/에 둡니다. 대화가 길어질수록 사용량이 빨리 줄어듭니다.
- **도구가 달라도:** 팀원마다 쓰는 AI 도구가 다를 수 있으니, 규칙은 CLAUDE.md · docs · PR 리뷰로 맞춥니다.

AI가 대신 못 하는 일 — 사람이 직접 챙길 것

- **외부 서비스 설정:** 카카오 앱 · 토스 개발자센터 · AWS 계정과 요금 설정 (G · Y단계)
- **팀 합의:** 역할 · 일정 · ERD · API 변경은 사람끼리 회의에서 정합니다.
- **판단과 설명:** 에러의 원인을 짚고, 발표에서 “이 코드가 왜 이렇게 동작하는지” 답할 수 있어야 합니다.

15 리스크와 대응

막히면 이 표부터 확인하세요. 마지막 열은 관련 단계입니다.

15.1 기술 리스크

#	문제	증상	대응	단계
1	카카오맵 무료 쿼터 대상 아님	활성화할 때 비즈월렛 연결 요구	팀 앱 하나만, 계정에서 처음 활성화한 앱 사용	G
2	카카오 도메인 미등록 · 좌표 순서	지도가 안 뜸 / 마커가 바다에 찍힘	localhost · 배포 도메인 모두 등록 / `x = 경도`, `y = 위도`	G · M
3	토스 키 조합 오류	인증되지 않은 키 오류	같은 세트의 클라이언트 · 시크릿 키, 테스트 · 라이브 섞지 않기, API 개별 연동 키 + 결제창	G · N
4	결제 금액 위조	화면에서 금액을 바꿔 결제	신청 시 서버 계산 금액 저장 → 승인 전 비교	N
5	승인 후 저장 실패	돈은 결제됐는데 접수 안 됨	승인 결과 로그, 저장 실패 시 결제 취소 API로 되돌림	N
6	위치를 못 받음	휴대폰에서 위치 권한 창이 안 뜸	HTTPS 터널 · 배포 HTTPS 사용, 권한 거부 안내 화면	R · Y
7	위치 전송 멈춤	화면이 꺼지면 마커가 안 움직임	시연 중 기사 화면 켜 두기, “마지막 위치 n초 전” 표시	R
8	SSE 인증 · 끊김	알림이 안 오거나 계속 재연결	`event-source-polyfill`, heartbeat, Nginx 버퍼링 끄기	R · Y
9	CORS · 토큰 만료	CORS 오류 / 일정 시간 뒤 모든 요청 실패	허용 출처 관리 / 인터셉터 재발급 후 재요청	H · K

#	문제	증상	대응	단계
10	이미지 · 업로드	`<img>` 401 / 휴대폰 사진 업로드 실패	파일 조회는 인증 없이 + UUID 이름 / multipart 한도 상향 (기본 1MB)	Q
11	동시 배차	같은 배송에 기사 두 명	`@Version` + 활성 배차 1건 검사	O
12	N+1 · 시간대	목록이 느림 / 9시간 차이	fetch join · `@EntityGraph` / 모든 곳 `Asia/Seoul`	S · I
13	키 유출 · AWS 요금	GitHub에 키 노출 / 예상 못한 청구	`.gitignore`, 노출 즉시 재발급 / Budgets 알림 · 발표 후 정리	G · Y
14	Spring Boot 버전 차이	인터넷 · AI 예제가 컴파일 안 됨	수업과 같은 3.x 고정, CLAUDE.md에 명시	H

15.2 팀 협업 리스크

#	문제	증상	대응
1	통합 지옥	6주차에 한꺼번에 합치다 충돌 폭발	작은 PR, 하루 1번 develop 받아오기, 3주차부터 매주 통합 시연
2	공통 파일 충돌	SecurityConfig · ErrorCode · 라우터에서 매번 충돌	공통 파일 관리자 지정(11.4), ErrorCode는 도메인별 구역
3	약속 지연	엔티티 · 메서드가 없어서 다른 사람이 대기	빈 구현 먼저 공유(2.4), 지연은 데일리 스크럼에서 즉시 공유
4	로컬 DB 불일치	“내 컴퓨터에선 되는데”	스키마 변경 공지 → DB 초기화 · `data.sql` 재실행 (11.5)
5	역할 불균형	한 사람에게 일이 몰리거나 진행이 멈춤	이슈 보드로 진행률 공유, 주간 회의에서 화면 일부 재분배
6	스타일 불일치	화면 · 코드가 사람마다 다름	공용 CLAUDE.md, 공통 컴포넌트, 짝 리뷰, 9주차 UI 점검
7	범위 과다	6주차에 MVP 미완성	MVP 먼저, 미루는 순서: 공지 · FAQ → 정산 → 리뷰 → 통계 → 위치 (10주차 버퍼)

16 시연 시나리오와 최종 체크리스트

16.1 시연 시나리오 (약 7분)

노트북 2대(고객 · 관리자)와 실제 휴대폰 1대(기사)로 진행하고, 화면 전환은 발표 담당자가 맡습니다. 결제는 테스트 키라 실제 돈이 나가지 않습니다.

1. 고객 로그인 → 배송 신청: 주소 2개 검색 → 지도에 마커 · 거리 → 화물 종류 '파손주의', 600kg → 1톤 추천, 예상 운임 98,500원 → 토스 테스트 결제 → 접수
2. 관리자 대시보드 "오늘 접수" · 매출 증가 확인 → 배차 화면에서 근무 일정 · 평점 기준 추천 기사에게 배정
3. 고객 화면에 "배차 완료" 알림이 실시간으로 뜸
4. 기사 휴대폰으로 배차 확인 → 상차 사진 촬영 → "상차 완료" → "운송 시작" (위치 공유 시작)
5. 고객 배송 상세 지도에서 기사 위치가 움직이는 것 확인
6. 기사 "도착" → 인수증 사진 없이 "배송 완료" 시도 → 막힘(예외 처리) → 사진 첨부 후 완료
7. 고객 타임라인 · 사진 확인 → 별점 5점 리뷰 작성
8. 관리자 기사 평점 · 대시보드 변화 확인 → 월 정산 생성 → 지급 처리 (취소 · 환불은 미리 만든 다른 건으로 시연)

16.2 최종 체크리스트

기능

- [ ] 역할 3개 가입 · 로그인 · 권한, 약관 동의
- [ ] 신청 → 결제 → 배차 → 완료 전체 흐름
- [ ] 결제 실패 · 재시도 · 30분 자동 취소 · 환불
- [ ] 주소 검색 · 지도 · 운임 자동 계산
- [ ] 사진 · 필수 사진 검사, 리뷰 · 평점
- [ ] 실시간 알림 · 기사 위치 추적
- [ ] 검색 · 대시보드 · 정산 · 공지 · FAQ · 문의

품질

- [ ] 운임 · 결제 검증 · 상태 전이 · 배차 · 정산 단위 테스트
- [ ] 권한 없는 접근은 403
- [ ] 기사 화면을 실제 휴대폰(HTTPS)으로 확인
- [ ] 4명의 화면 스타일 통일, 에러 메시지 정리
- [ ] 시연용 더미 데이터 · 계정 준비

배포 · 보안

- [ ] HTTPS 배포 URL에서 시연 시나리오 통과
- [ ] 카카오 도메인 · 토스 successUrl 배포 주소로
- [ ] 키 · 비밀번호가 Git에 없음
- [ ] AWS 예산 알림 · 무료 종료일 확인, 발표 후 정리 계획

문서 · 발표 · 협업

- [ ] README (팀 · 구성도 · ERD · 실행 방법)
- [ ] Swagger · API 명세 최신
- [ ] 각자 트러블슈팅 2개 이상, decisions.md
- [ ] PR 리뷰 기록 · 이슈 보드 정리
- [ ] 발표 자료 각자 파트, 시연 영상, 예상 질문

17 참고 자료

외부 서비스 정책은 바뀔 수 있습니다. 아래 내용은 2026년 11월 2일에 확인했으니 G단계 · Y단계에서 한 번 더 확인하세요.

- **카카오맵 API 신규 기능 및 무료 쿼터 운영 방식 변경 안내** (카카오 데브톡) — 2026년 7월 21일부터 심사 없이 활성화, 계정당 첫 번째 활성화 앱에만 무료 쿼터
  `https://devtalk.kakao.com/t/api-notice-on-new-kakao-map-api-features-and-free-quota-policy/150222`
- **카카오디벨로퍼스 쿼터 안내**
  `https://developers.kakao.com/docs/latest/ko/getting-started/quota`
- **카카오모빌리티 디벨로퍼스 요금 · 무료 쿼터** — 자동차 길찾기 API (비상업 용도는 무료 쿼터 안에서 별도 제휴 없이 사용)
  `https://developers.kakaomobility.com/price/`
- **토스페이먼츠에 가장 많이 물어보는 질문 세 가지** (토스페이먼츠 블로그) — 계약 없이 회원가입만으로 테스트 키 발급, 테스트 키로는 실제 결제가 일어나지 않음
  `https://www.tosspayments.com/blog/articles/22608`
- **결제 연동, 코드 없이 시작하기 — 샌드박스** (토스페이먼츠 블로그)
  `https://www.tosspayments.com/blog/articles/introducing-sandbox`
- **Geolocation on secure contexts only** (Chrome for Developers) — 위치 API는 HTTPS 또는 localhost에서만
  `https://developer.chrome.com/blog/geolocation-on-secure-contexts-only`
- **AWS 프리 티어 FAQ** — 무료 플랜 6개월 또는 크레딧 소진 시 종료
  `https://aws.amazon.com/ko/free/free-tier-faqs/`
- **AWS 프리 티어 업데이트** (AWS 한국 블로그) — 신규 고객 최대 200달러 크레딧
  `https://aws.amazon.com/ko/blogs/korea/aws-free-tier-update-new-customers-can-get-started-and-explore-aws-with-up-to-200-in-credits`
- **Spring Boot 4.0 개요** — 2025년 11월 출시, Spring Framework 7 · Security 7 · Jackson 3 기반
  `https://versionlog.com/spring-boot/4.0/`